

4.1 预训练语言模型

陈露

上海交通大学计算机科学与工程系
2023年秋季学期

- ▶ 词向量回顾
- ▶ 上下文相关词向量早期发展
 - ▶ CoVe
 - ▶ ELMo
- ▶ 典型预训练语言模型及其分类
 - ▶ BERT
 - ▶ GPT
 - ▶ BART
 - ▶ 其他
- ▶ 预训练语言模型的发展和前景

- ▶ 出发点:
 - ▶ 理解自然语言需要对单词进行建模
 - ▶ 单热向量维度爆炸且不能反映词间关系

- ▶ 出发点：
 - ▶ 理解自然语言需要对单词进行建模
 - ▶ 单热向量维度爆炸且不能反映词间关系
- ▶ 经典词向量方法：
 - ▶ CBOW：邻域词预测中心词
 - ▶ Skip-gram：中心词预测邻域词

词向量回顾

- ▶ {水果包括苹果、香蕉、橙子、橘子等
新出的苹果手机没有华为手机好用
- ▶ {一朝被蛇咬，十年怕井绳
面朝大海，春暖花开
- ▶ 干一行行一行，一行行行行行

- ▶ { 水果包括苹果、香蕉、橙子、橘子等
 { 新出的苹果手机没有华为手机好用
- ▶ { 一朝被蛇咬，十年怕井绳
 { 面朝大海，春暖花开
- ▶ 干一行行一行，一行行行行行
- ▶ 传统词向量的问题：
 - ▶ 静态——上下文无关
 - ▶ 使用时需要引入复杂结构来建模上下文信息
 - ▶ 对于不同任务有时需要分别设计上下文建模模型

- ▶ 预训练向量 $\Rightarrow$ 预训练上下文建模模型
- ▶ 大规模数据预训练复杂模型 + 有限的领域特定数据微调简单模型
- ▶ 意义：
 - ▶ 极大的开发和揭示了大规模预训练的潜力
 - ▶ 极大的简化和统一了上下文信息建模的模型结构

上下文相关词向量的早期发展

Context Vector (CoVe)

- ▶ 第一次提出上下文相关词向量
- ▶ 受启发于CV中在ImageNet上预训练模型后在其他小数据集上使用

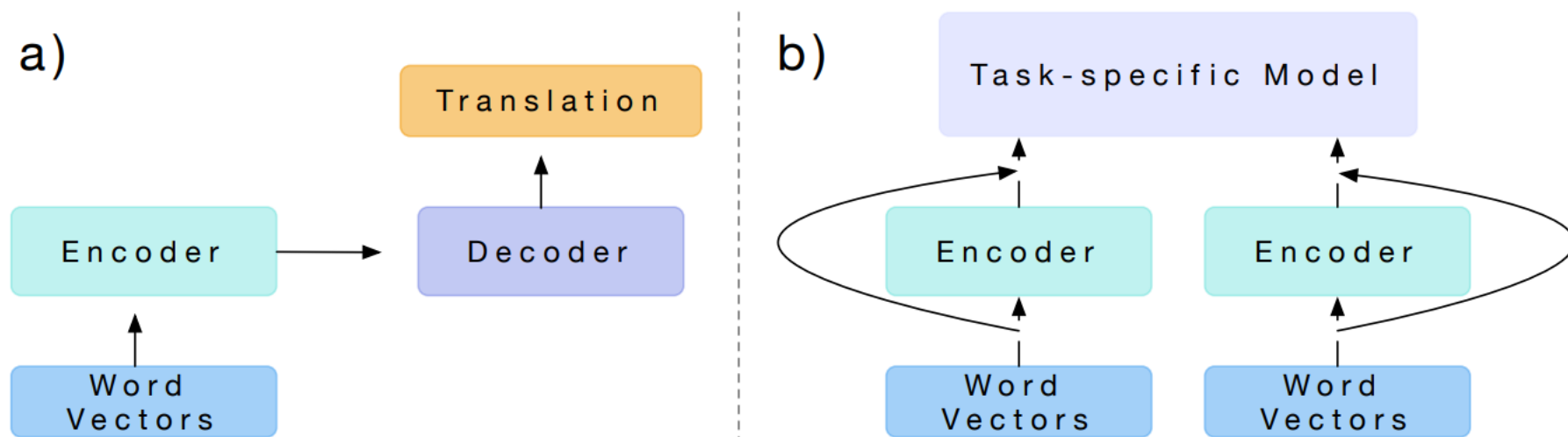


Figure 1: We a) train a two-layer, bidirectional LSTM as the encoder of an attentional sequence-to-sequence model for machine translation and b) use it to provide context for other NLP models.

上下文相关词向量的早期发展

Context Vector (CoVe)

- ▶ 第一次提出上下文相关词向量
- ▶ 受启发于CV中在ImageNet上预训练模型后在其他小数据集上使用
- ▶ 问题：
 - ▶ 预训练过程使用的是有监督数据，需要大量额外人工标注

基本任务：已知前文预测下一个字

- ▶ 输入：单词序列 $w_1, w_2, \dots, w_{k-1}$
- ▶ 输出：对下一个单词预测的概率 $P(w_k | w_1, \dots, w_{k-1})$

或者说：推测句子的合理程度

- ▶ $\prod_{k=1}^K P(w_k | w_1, \dots, w_{k-1}) = P(w_1, w_2, \dots, w_K)$

神经网络语言模型的训练任务

- ▶ 序列到序列建模，整句输入网络，每个词元 (Token) 的位置预测下一个词元

基本任务：已知前文预测下一个字

- ▶ 输入：单词序列 $w_1, w_2, \dots, w_{k-1}$
- ▶ 输出：对下一个单词预测的概率 $P(w_k | w_1, \dots, w_{k-1})$

或者说：推测句子的合理程度

- ▶ $\prod_{k=1}^K P(w_k | w_1, \dots, w_{k-1}) = P(w_1, w_2, \dots, w_K)$

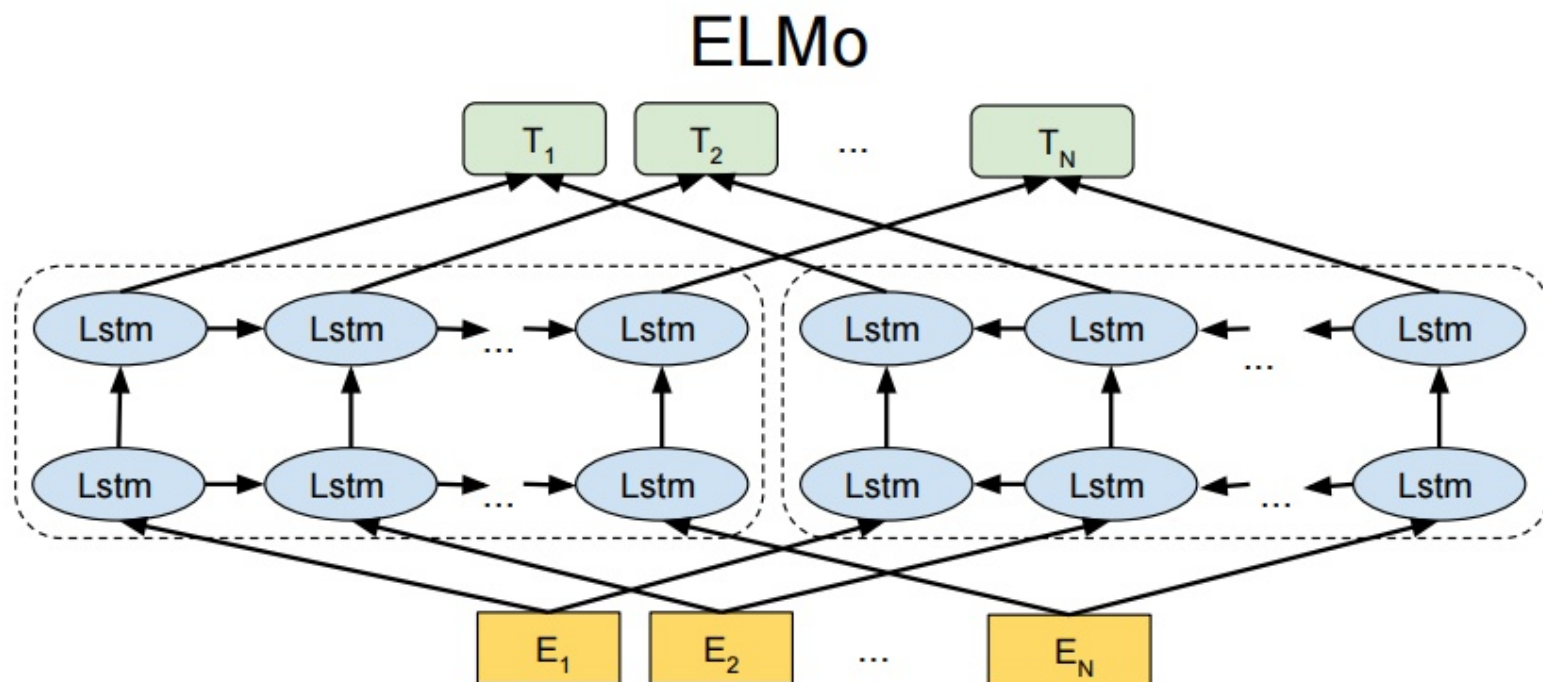
神经网络语言模型的训练任务

- ▶ 序列到序列建模，整句输入网络，每个词元 (Token) 的位置预测下一个词元
- ▶ 可以**自我监督**，不需要额外的标注！

上下文相关词向量的早期发展

Embeddings from Language Models (ELMo)

- ▶ 利用语言模型的自监督特性，不需要额外标注
- ▶ 使用双向语言模型，建模双向的依赖关系



Peters, Matthew E., et al. "Deep contextualized word representations". *NAACL* 2018.

上下文相关词向量的早期发展

Embeddings from Language Models (ELMo)

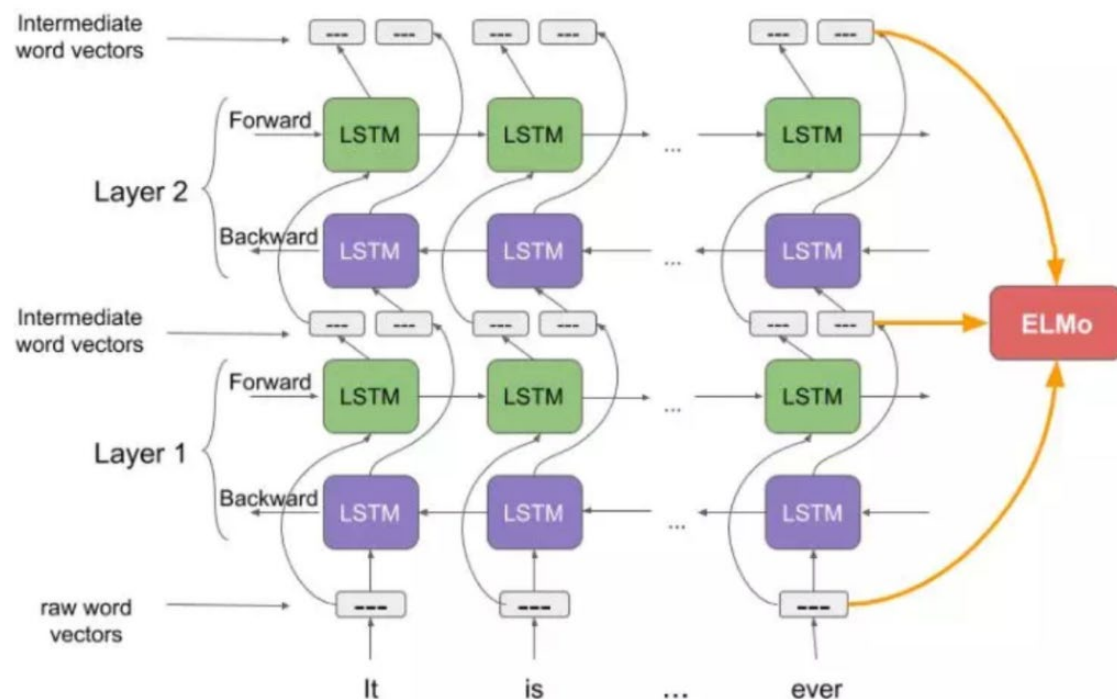
- ▶ 利用语言模型的自监督特性，不需要额外标注
- ▶ 使用双向语言模型，建模双向的依赖关系

$$\sum_{k=1}^N (\log p(t_k \mid t_1, \dots, t_{k-1}; \Theta_x, \vec{\Theta}_{LSTM}, \Theta_s) \\ + \log p(t_k \mid t_{k+1}, \dots, t_N; \Theta_x, \overleftarrow{\Theta}_{LSTM}, \Theta_s)).$$

上下文相关词向量的早期发展

Embeddings from Language Models (ELMo)

- ▶ 利用语言模型的自监督特性，不需要额外标注
- ▶ 使用双向语言模型，建模双向的依赖关系
- ▶ 加权平均所有中间层向量得到动态词向量



Peters, Matthew E., et al. "Deep contextualized word representations". *NAACL* 2018.

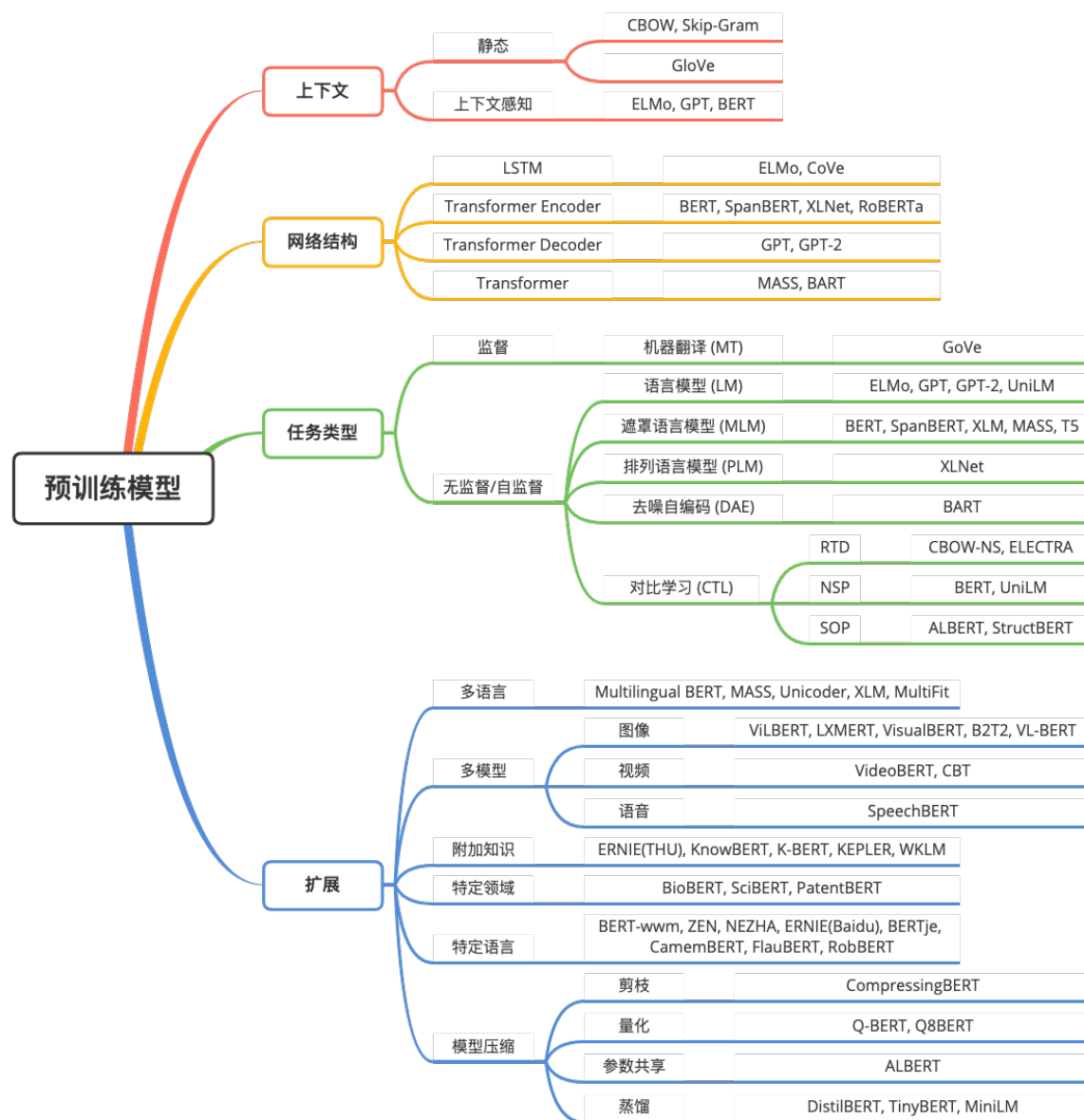
上下文相关词向量和语言模型

- ▶ 上下文相关词向量需要大量的有监督数据
- ▶ 语言模型提供的自监督思想不需要人工标注
- ▶ 一拍即合！

上下文相关词向量和语言模型

- ▶ 上下文相关词向量需要大量的有监督数据
 - ▶ 语言模型提供的自监督思想不需要人工标注
 - ▶ 一拍即合！
-
- ▶ 其实传统的词向量（CBOW, SG等）的训练也可以理解为对整句话的联合概率进行建模
$$\begin{aligned} P(w_1, w_2, w_3) &= P(w_2|w_1, w_3)P(w_1, w_3) \\ &= P(w_1, w_3|w_2)P(w_2) \end{aligned}$$
 - ▶ 即也是使用的语言模型思想进行训练

典型预训练语言模型及其分类



Qiu, Xipeng, et al. "Pre-trained models for natural language processing: A survey." *Science China Technological Sciences* (2020): 1-26.

- ▶ 词向量回顾
- ▶ 上下文相关词向量早期发展
 - ▶ CoVe
 - ▶ ELMo
- ▶ 典型预训练语言模型及其分类
 - ▶ BERT
 - ▶ GPT
 - ▶ BART
 - ▶ 其他
- ▶ 预训练语言模型的发展和前景

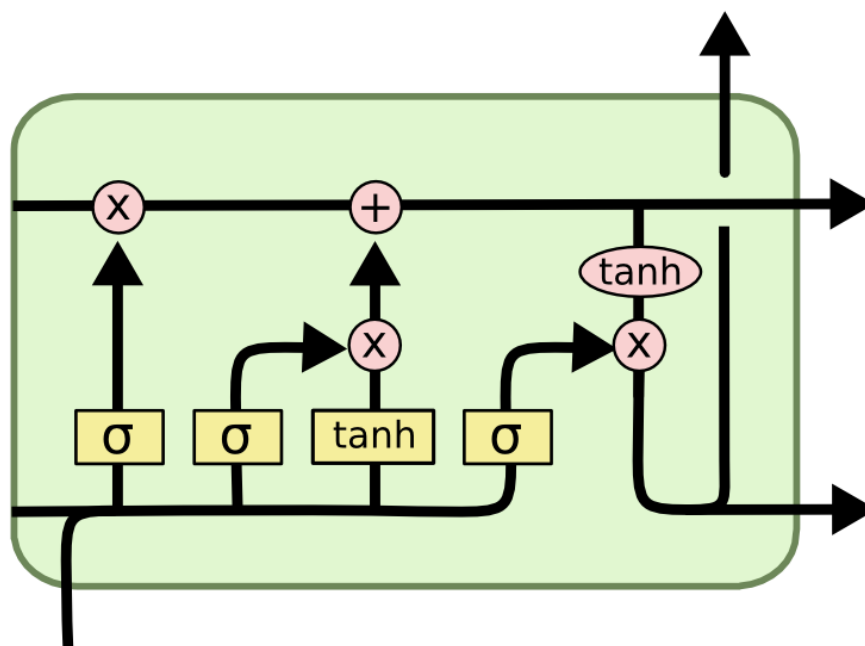
双向自编码模型：BERT

- ▶ 一个高效的预训练语言模型的要求：
 - ▶ 强大的建模能力
 - ▶ 有效建模双向的上下文信息
 - ▶ 能够利用大规模无监督数据

双向自编码模型：BERT

建模能力：LSTM v.s. Transformer

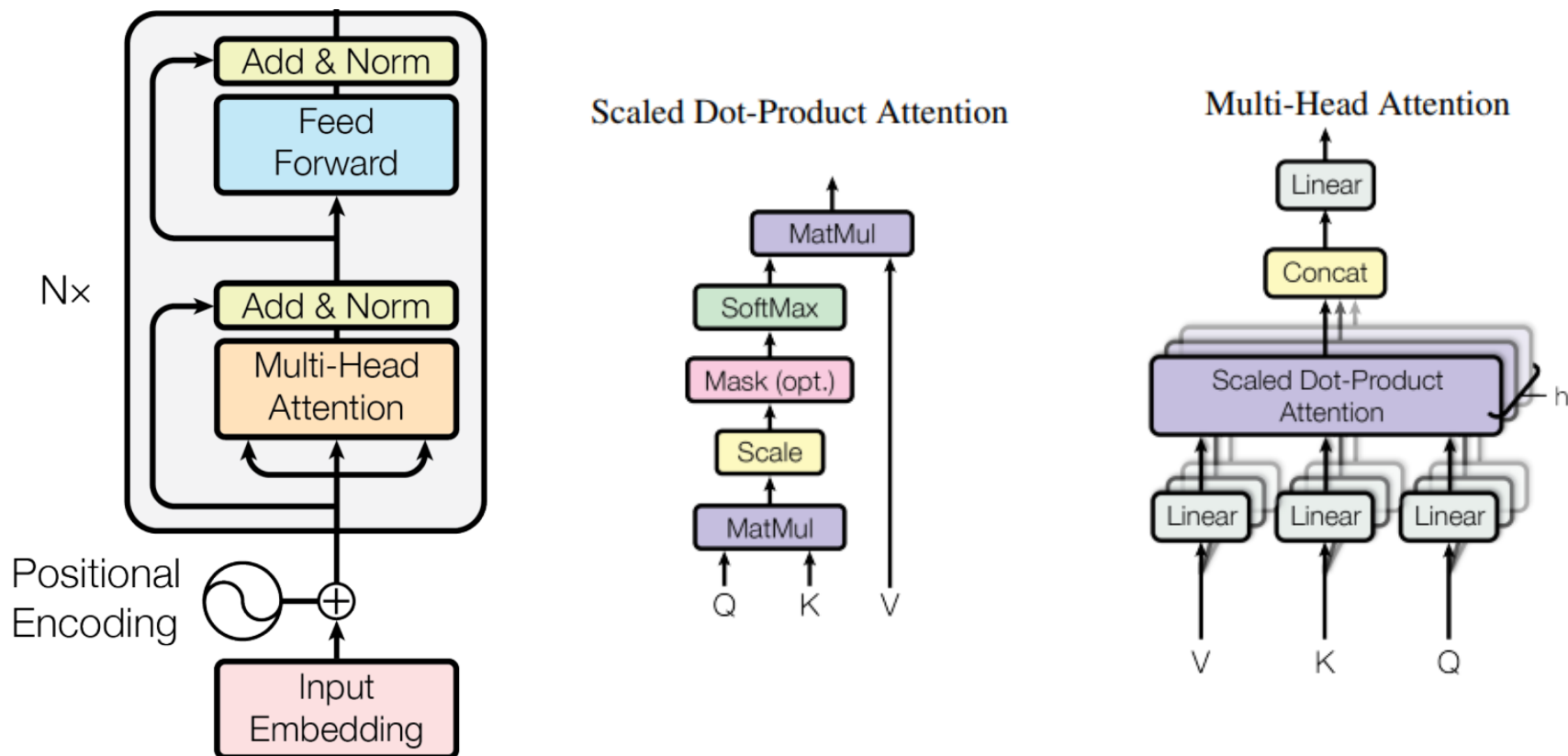
- ▶ ELMo中的Bi-LSTM的建模能力有限



双向自编码模型：BERT

建模能力：LSTM v.s. Transformer

► 使用基于Attention的Transformer结构

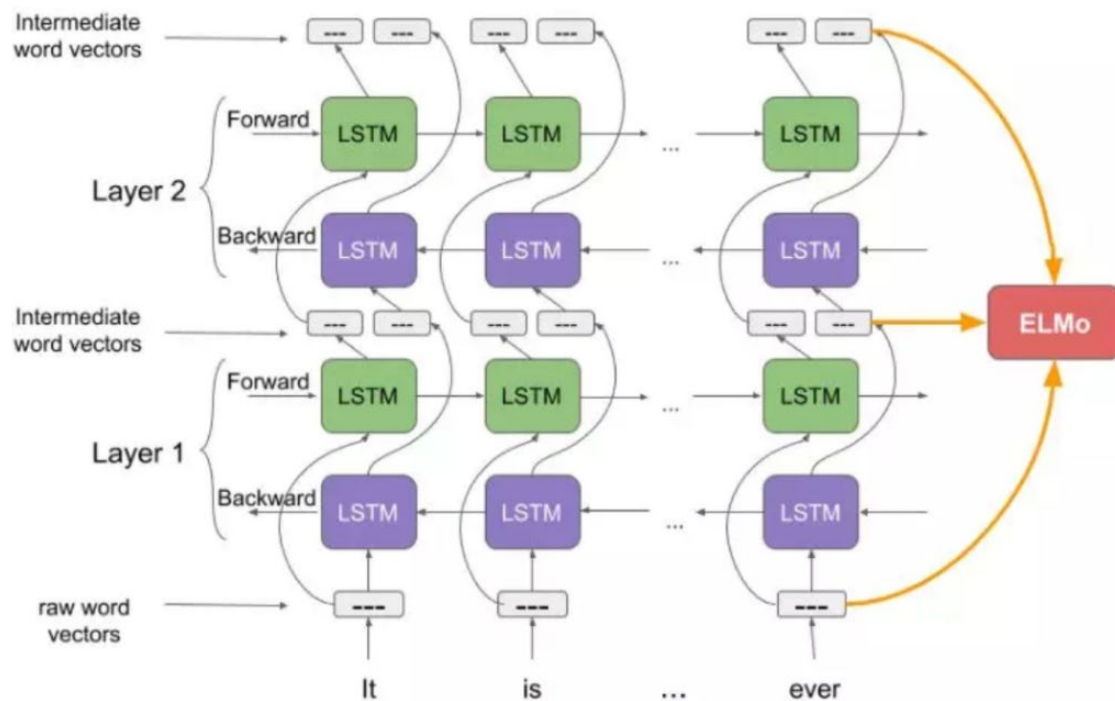


Vaswani, Ashish, et al. "Attention is all you need." Proceedings of the 31st International Conference on Neural Information Processing Systems. 2017.

双向自编码模型：BERT

双向上下文信息：自回归 v.s. 自编码

- ▶ 使用基于Attention的Transformer结构
- ▶ 自回归模型难以有效建模双向上下文信息

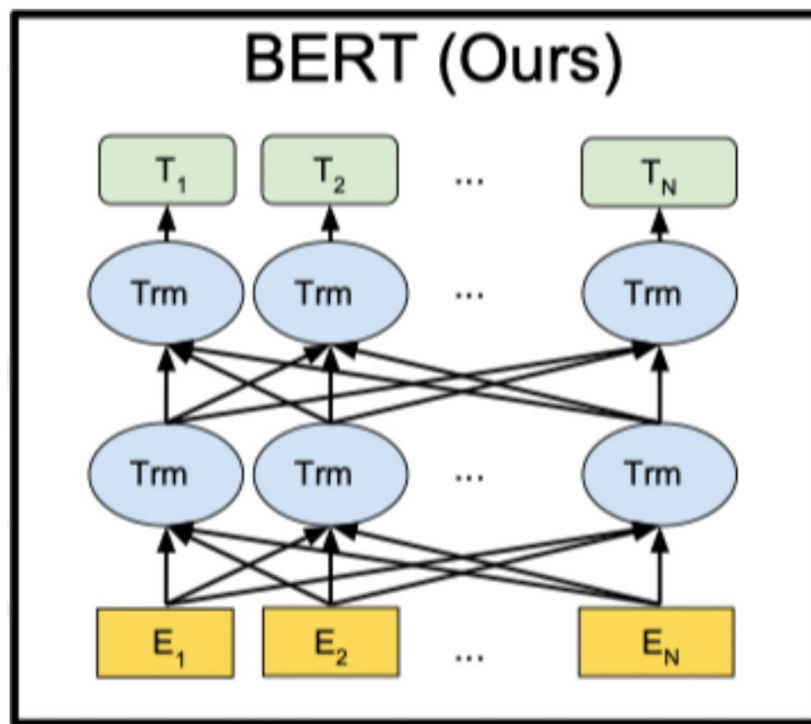


Peters, Matthew E., et al. "Deep contextualized word representations". *NAACL* 2018.

双向自编码模型：BERT

双向上下文信息：自回归 v.s. 自编码

- ▶ 使用基于Attention的Transformer结构
- ▶ 仅使用Transformer Encoder结构（即自编码结构）

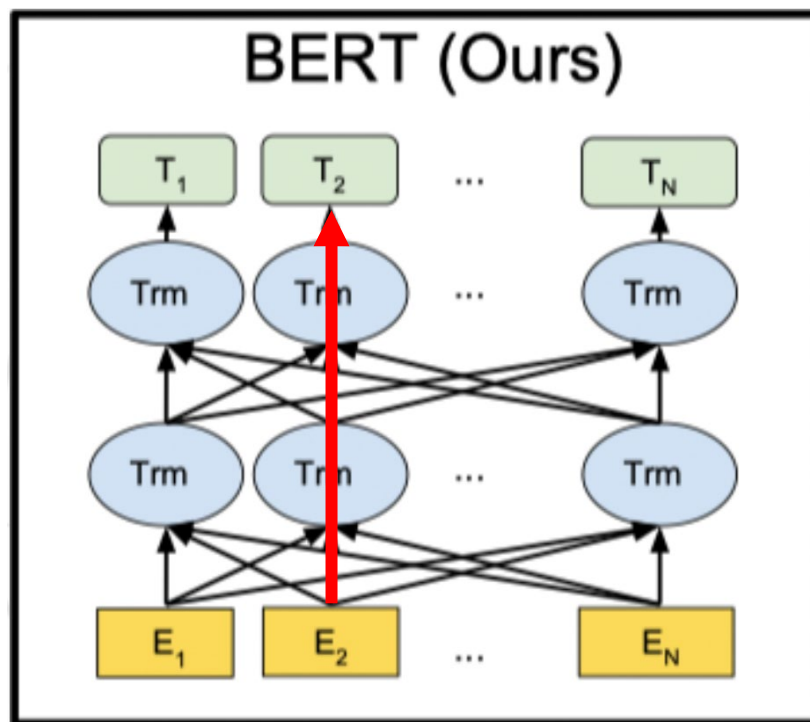


Kenton, Jacob Devlin Ming-Wei Chang, and Lee Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT. 2019.

双向自编码模型：BERT

自监督任务：LM v.s. MLM

- ▶ 使用基于Attention的Transformer结构
- ▶ 仅使用Transformer Encoder结构（即自编码结构）
- ▶ 双向建模会导致信息泄露

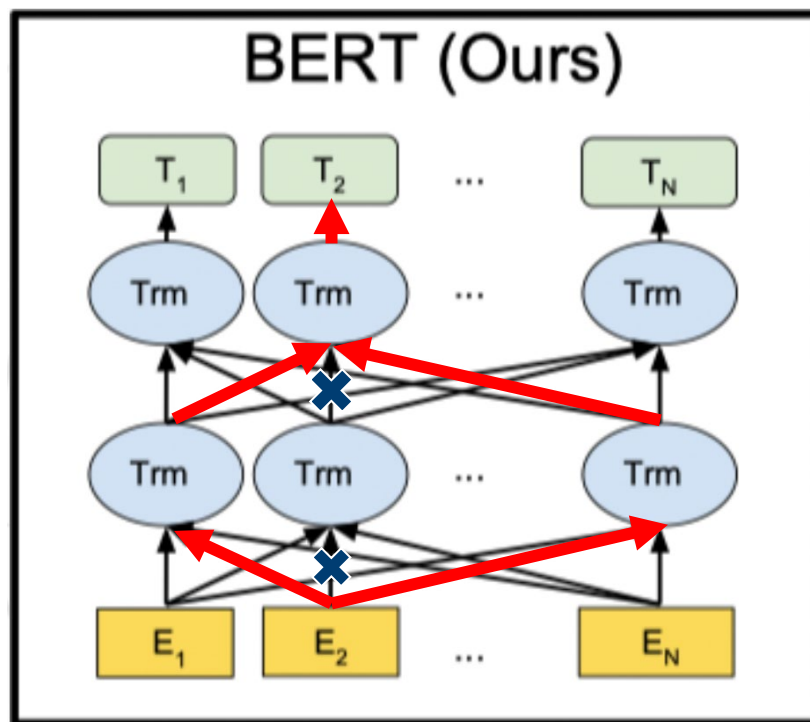


Kenton, Jacob Devlin Ming-Wei Chang, and Lee Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT. 2019.

双向自编码模型：BERT

自监督任务：LM v.s. MLM

- ▶ 使用基于Attention的Transformer结构
- ▶ 仅使用Transformer Encoder结构（即自编码结构）
- ▶ 双向建模会导致信息泄露



Kenton, Jacob Devlin Ming-Wei Chang, and Lee Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT. 2019.

双向自编码模型：BERT

自监督任务：LM v.s. MLM

- ▶ 使用基于Attention的Transformer结构
- ▶ 仅使用Transformer Encoder结构（即自编码结构）
- ▶ 新的预训练任务：掩码语言模型（Masked Language Model, MLM）

双向自编码模型：BERT

自监督任务：LM v.s. MLM

Use the output of the masked word's position to predict the masked word

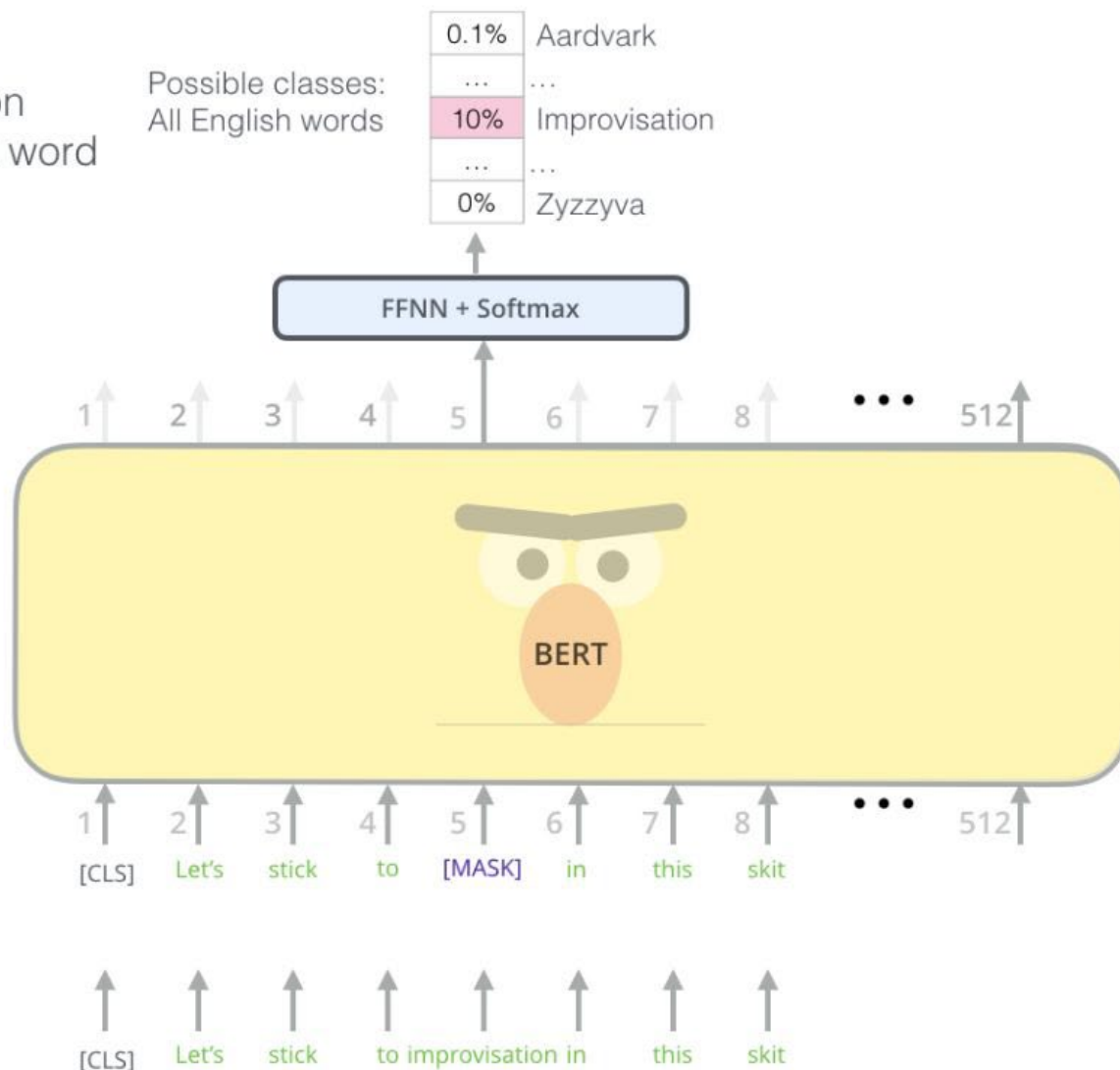
Possible classes:
All English words

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzzzyva

FFNN + Softmax

Randomly mask
15% of tokens

Input



By Jay Alammar's Blog "The Illustrated BERT, ELMo, and co. (How NLP Cracked Transfer Learning)"

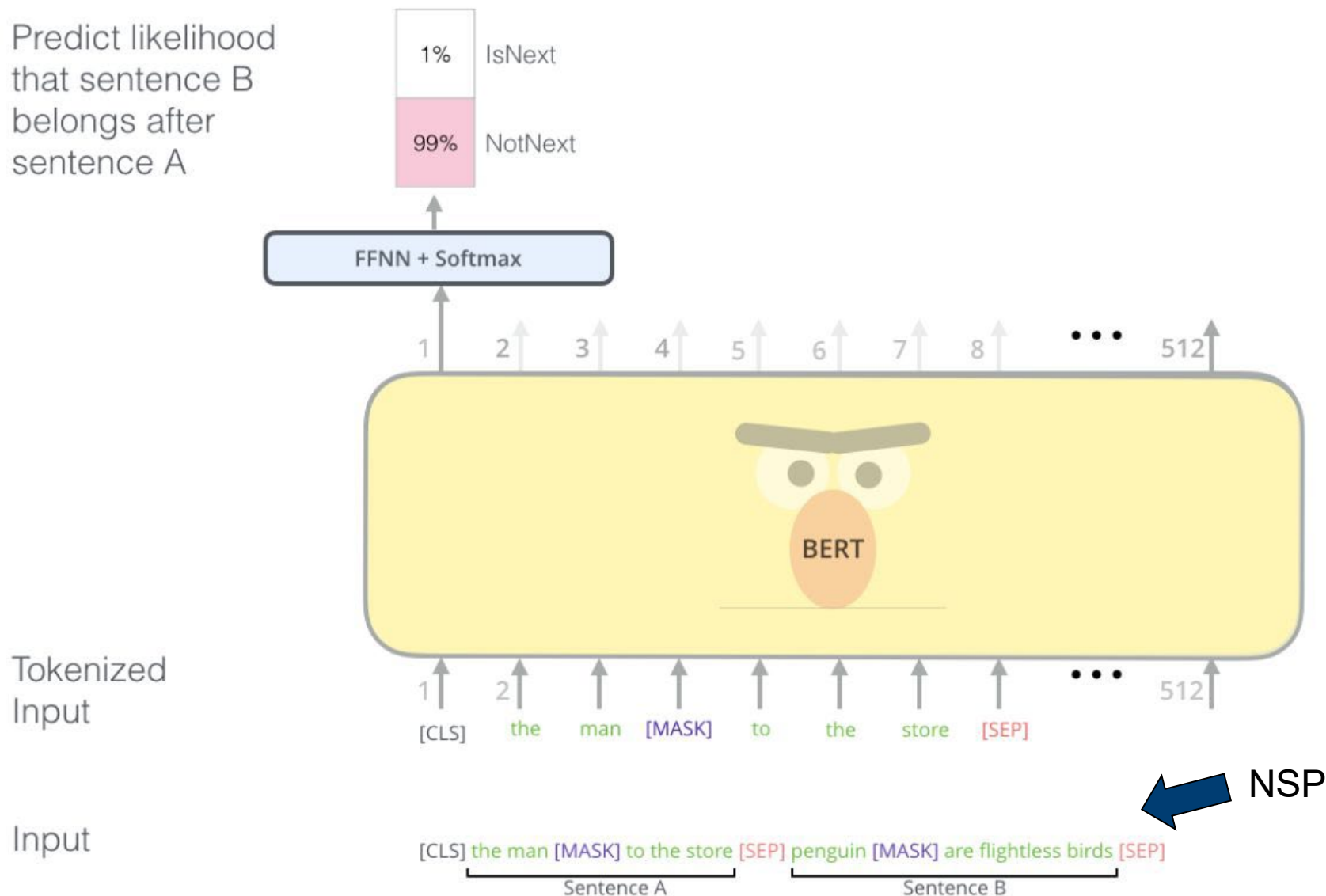
双向自编码模型：BERT

自监督任务：NSP

- ▶ 使用基于Attention的Transformer结构
- ▶ 仅使用Transformer Encoder结构（即自编码结构）
- ▶ 新的预训练任务：掩码语言模型（Masked Language Model, MLM）
- ▶ 另外，还引入了NSP(Next Sentence Prediction)任务来引入句间关系的建模

双向自编码模型：BERT

自监督任务：NSP

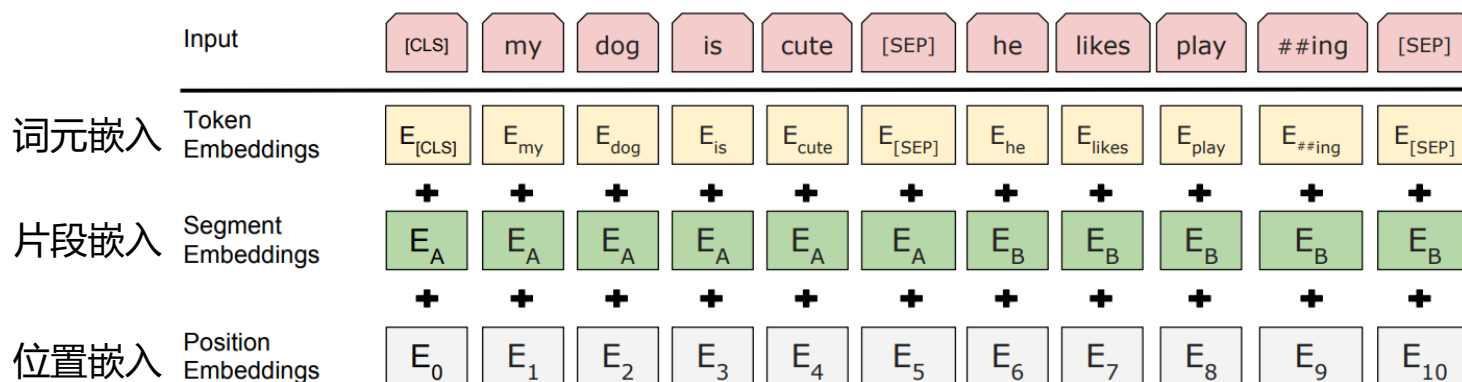


By Jay Alammar's Blog "The Illustrated BERT, ELMo, and co. (How NLP Cracked Transfer Learning)"

双向自编码模型：BERT

模型设计总结

- ▶ 使用Transformer Encoder的自编码模型进行建模
- ▶ 输入组成：
 - ▶ 词元嵌入 + 片段嵌入 + 位置嵌入
 - ▶ [CLS] & [SEP]
 - ▶ BPE (Token)



Kenton, Jacob Devlin Ming-Wei Chang, and Lee Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT. 2019.

双向自编码模型：BERT

模型设计总结

- ▶ 使用Transformer Encoder的自编码模型进行建模
- ▶ 输入组成：
 - ▶ 词元嵌入 + 片段嵌入 + 位置嵌入
 - ▶ [CLS] & [SEP]
 - ▶ BPE (Token)
- ▶ 预训练任务：
 - ▶ MLM
 - ▶ 掩蔽15%的词元：10%保留、10%随机替换、80%[MASK]
 - ▶ NSP
 - ▶ 50%正例+50%负例

双向自编码模型：BERT

参数量及训练量

- ▶ $BERT_{BASE}$
 - ▶ $L = 12, H = 768, A = 12$
 - ▶ 参数总量100M
- ▶ $BERT_{LARGE}$
 - ▶ $L = 24, H = 1024, A = 16$
 - ▶ 参数总量340M
- ▶ 在总词数3.3B的数据集上训练了40轮
- ▶ $BERT_{BASE}$ 使用16张TPU, $BERT_{LARGE}$ 使用64张TPU
- ▶ 总训练时间为4天

Kenton, Jacob Devlin Ming-Wei Chang, and Lee Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT. 2019.

双向自编码模型：BERT

预训练微调使用范式

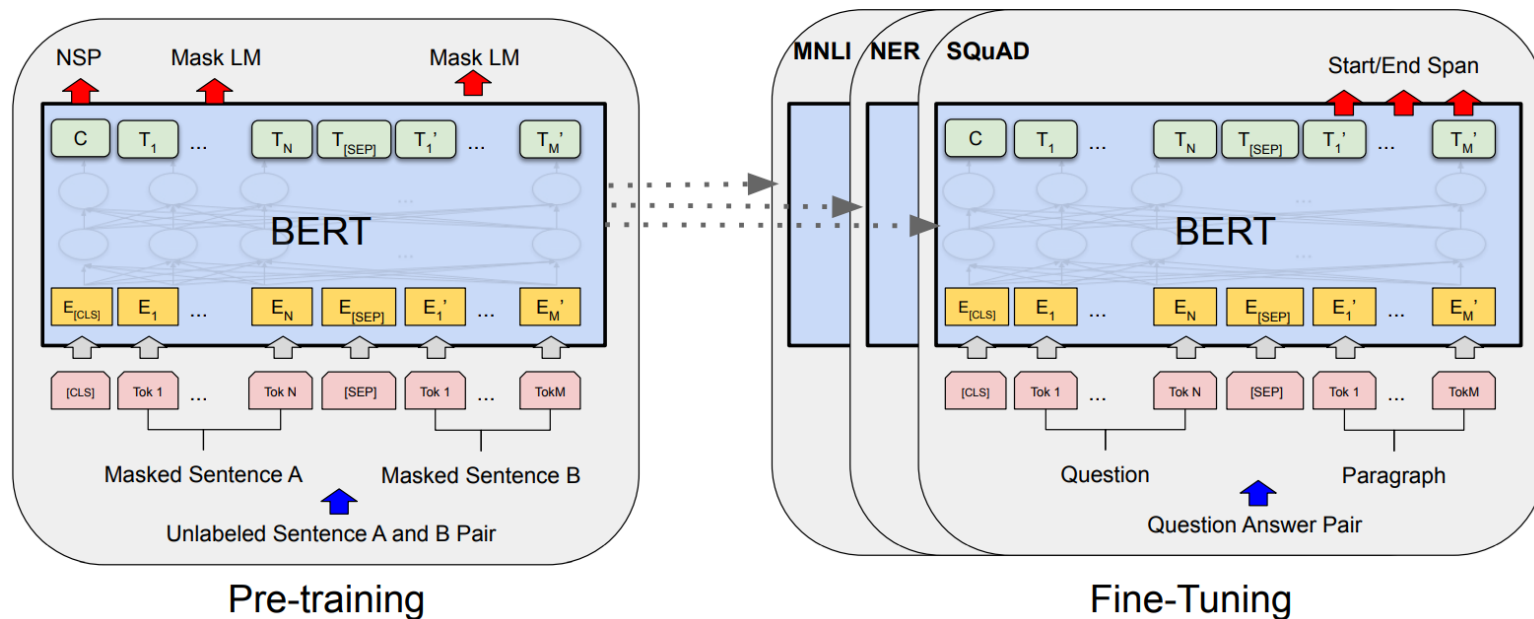
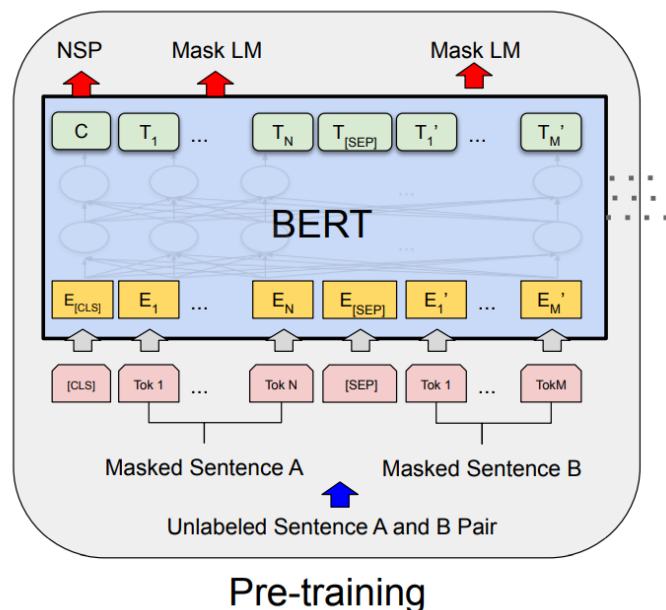


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

双向自编码模型：BERT

预训练微调使用范式

► 词向量在哪？

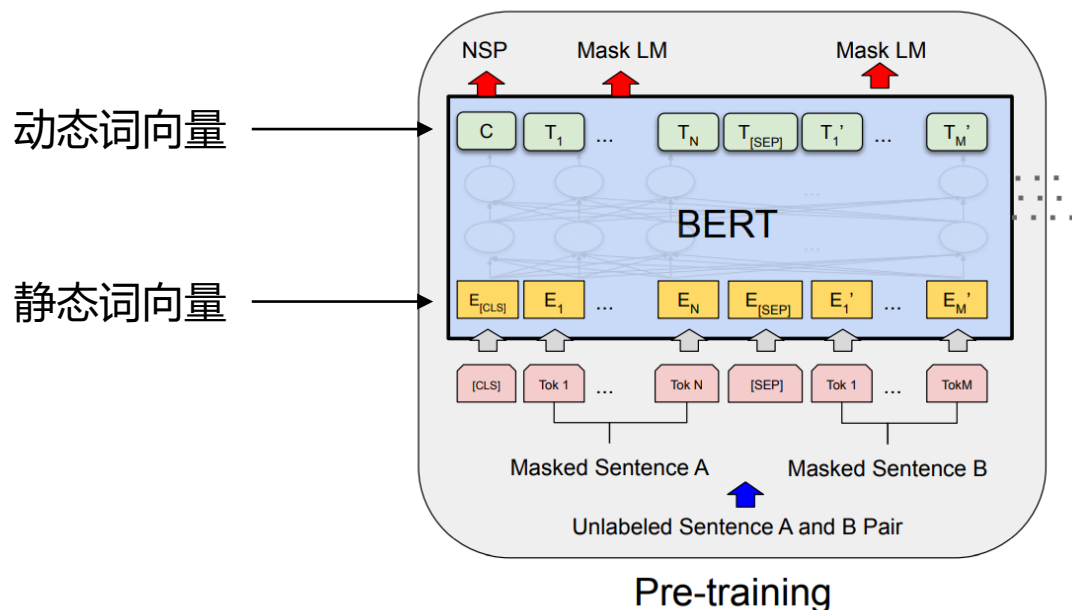


Kenton, Jacob Devlin Ming-Wei Chang, and Lee Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT. 2019.

双向自编码模型：BERT

预训练微调使用范式

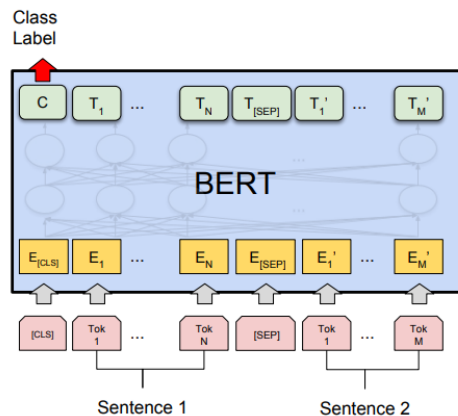
- ▶ 词向量在哪？
 - ▶ 静态词向量：输入中的Token Embedding
 - ▶ 动态词向量：线性分类层前的隐向量
- ▶ 下游任务可以直接改变线性分类层结构即可



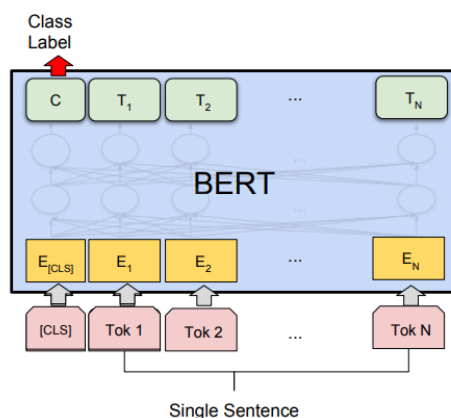
Kenton, Jacob Devlin Ming-Wei Chang, and Lee Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT. 2019.

双向自编码模型：BERT

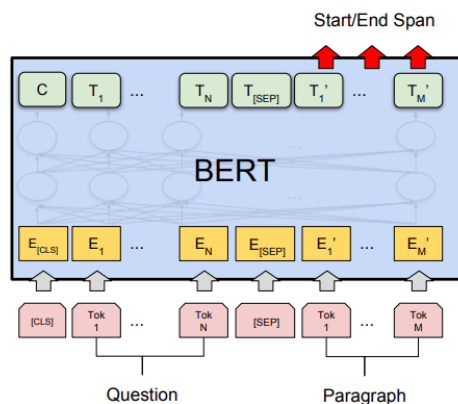
在下游任务上的微调：示例



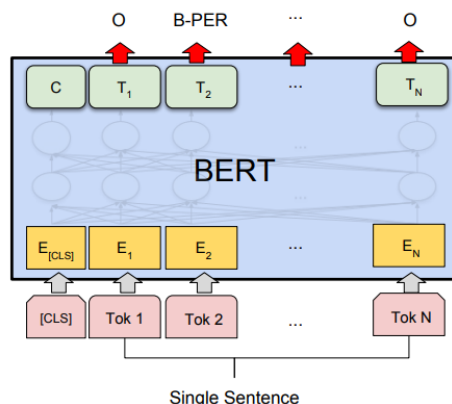
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



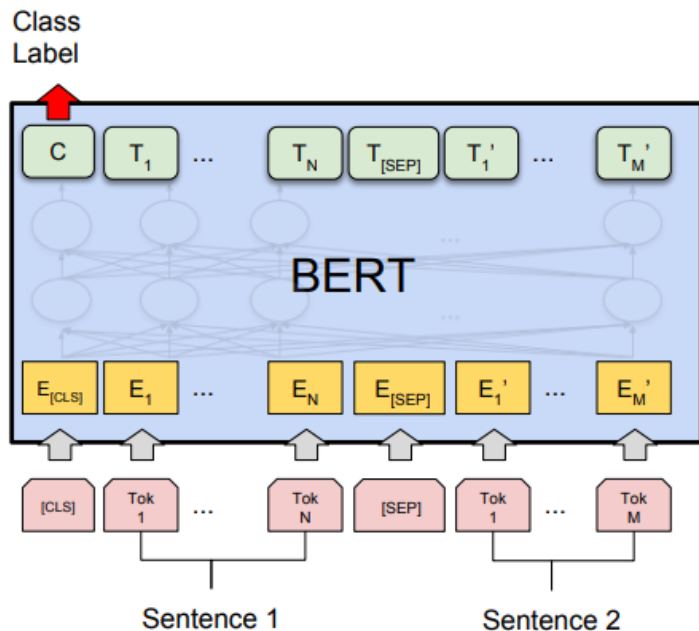
(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Kenton, Jacob Devlin Ming-Wei Chang, and Lee Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT. 2019.

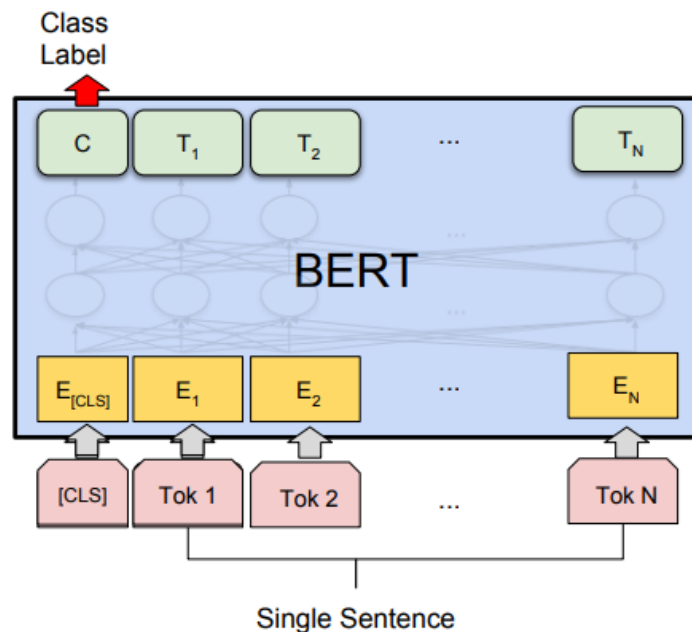
双向自编码模型：BERT

在下游任务上的微调：示例

- 只需要将最后的线性分类层换成相应的维度即可



(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG

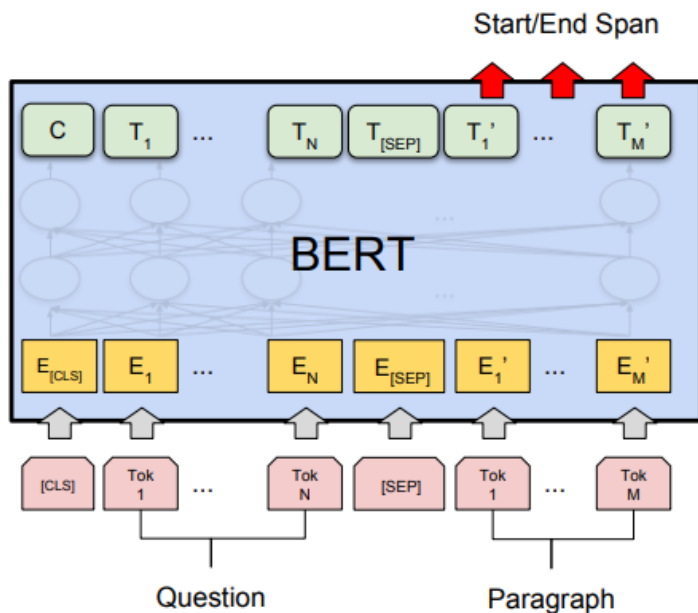


(b) Single Sentence Classification Tasks:
SST-2, CoLA

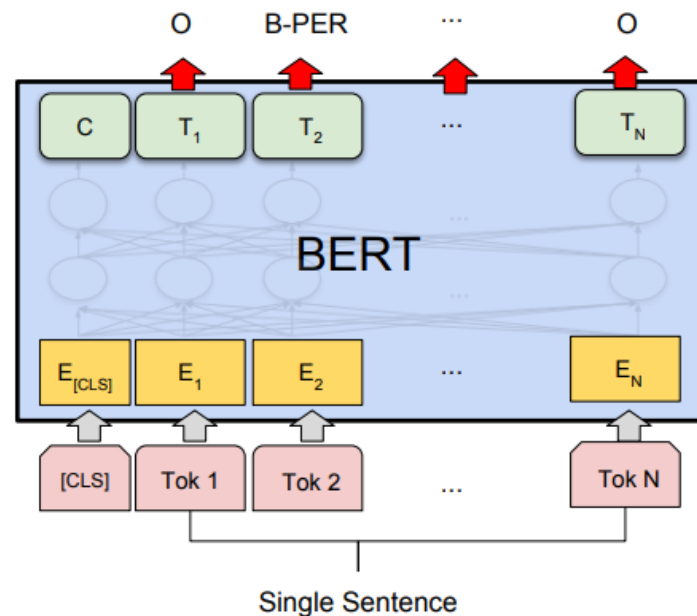
双向自编码模型：BERT

在下游任务上的微调：示例

- 只需要将最后的线性分类层换成相应的维度即可



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

双向自编码模型：BERT

在下游任务上的微调：GLUE结果

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The “Average” column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.⁸ BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

双向自编码模型：BERT

在下游任务上的微调：SQuAD结果

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
BERT _{BASE} (Single)	80.8	88.5	-	-
BERT _{LARGE} (Single)	84.1	90.9	-	-
BERT _{LARGE} (Ensemble)	85.8	91.8	-	-
BERT _{LARGE} (Sgl.+TriviaQA)	84.2	91.1	85.1	91.8
BERT _{LARGE} (Ens.+TriviaQA)	86.2	92.2	87.4	93.2

Table 2: SQuAD 1.1 results. The BERT ensemble is 7x systems which use different pre-training checkpoints and fine-tuning seeds.

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	86.3	89.0	86.9	89.5
#1 Single - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
Published				
unet (Ensemble)	-	-	71.4	74.9
SLQA+ (Single)	-	-	71.4	74.4
Ours				
BERT _{LARGE} (Single)	78.7	81.9	80.0	83.1

Table 3: SQuAD 2.0 results. We exclude entries that use BERT as one of their components.

双向自编码模型：BERT

- ▶ 优缺点：
 - ▶ 在自然语言理解任务上表现突出
 - ▶ 难以解决自然语言生成问题
 - ▶ [MASK]
 - ▶ 预训练和微调阶段的输入空间分布不匹配
 - ▶ 预训练过程收敛较慢，不能掩蔽过多词元
 - ▶ 假设被掩蔽词元是独立的，只能进行联合概率的有偏估计

双向自编码模型：BERT

- ▶ 优缺点：
 - ▶ 在自然语言理解任务上表现突出
 - ▶ 难以解决自然语言生成问题
 - ▶ [MASK]
 - ▶ 预训练和微调阶段的输入空间分布不匹配
 - ▶ 预训练过程收敛较慢，不能掩蔽过多词元
 - ▶ 假设被掩蔽词元是独立的，只能进行联合概率的有偏估计

- ▶ 直观理解独立性假设的问题：词组

New York is a city.

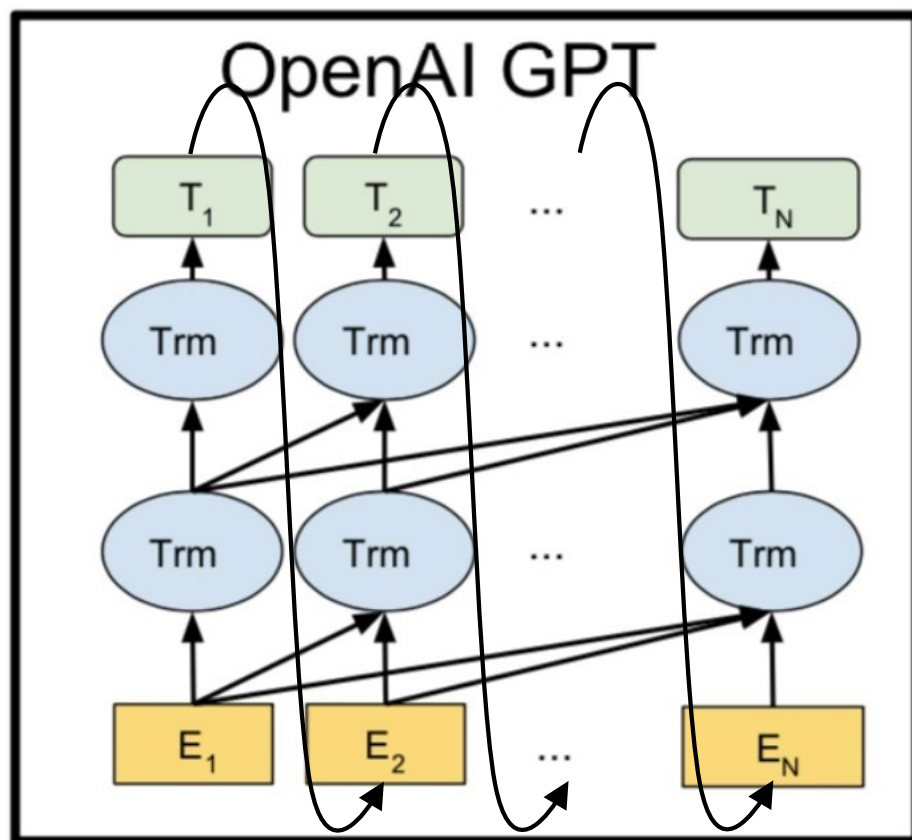
- ▶ 当New和York同时被掩蔽时

$$p(\text{New}, \text{York}) = p(\text{New}) \times p(\text{York}|\text{New}) \neq p(\text{New}) \times p(\text{York})$$

单向自回归模型：GPT

结构

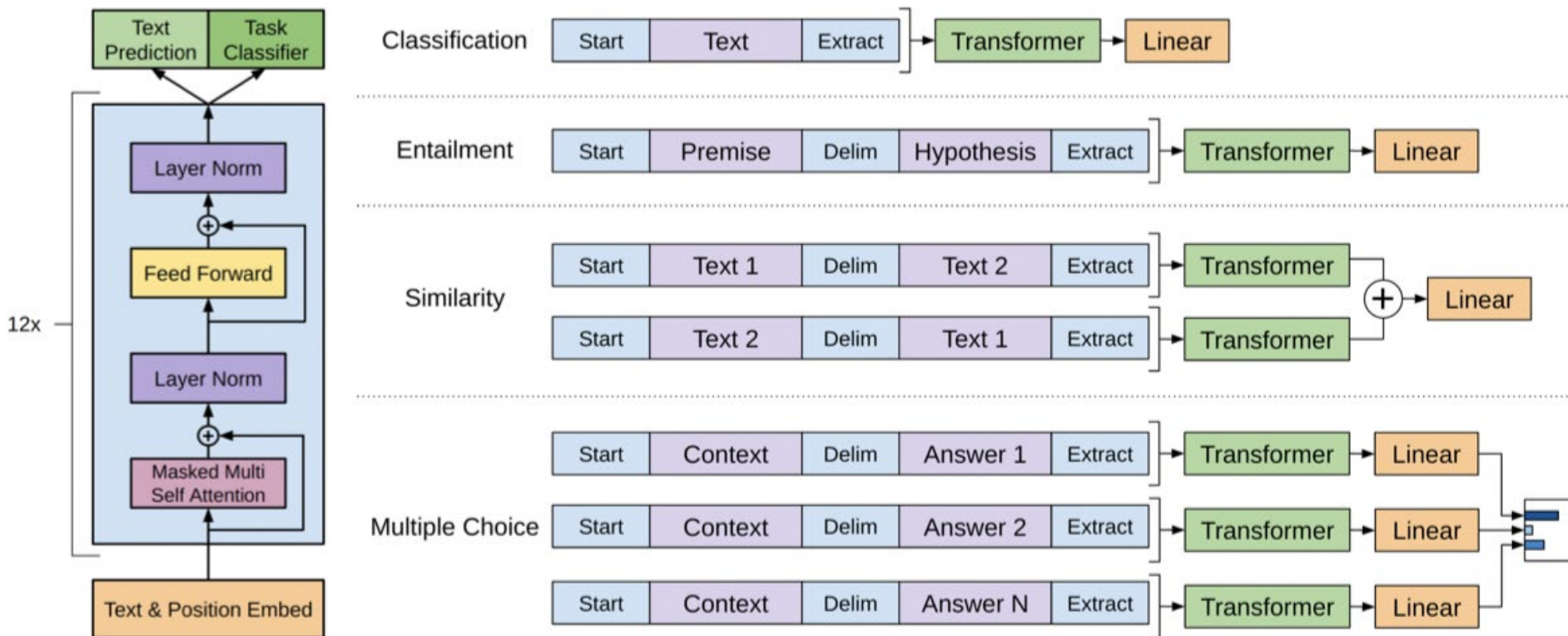
- ▶ 直接使用语言模型任务进行预训练
- ▶ 采用类Transformer Decoder结构（即自回归结构）



Kenton, Jacob Devlin Ming-Wei Chang, and Lee Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT. 2019.

单向自回归模型：GPT

下游任务使用方法



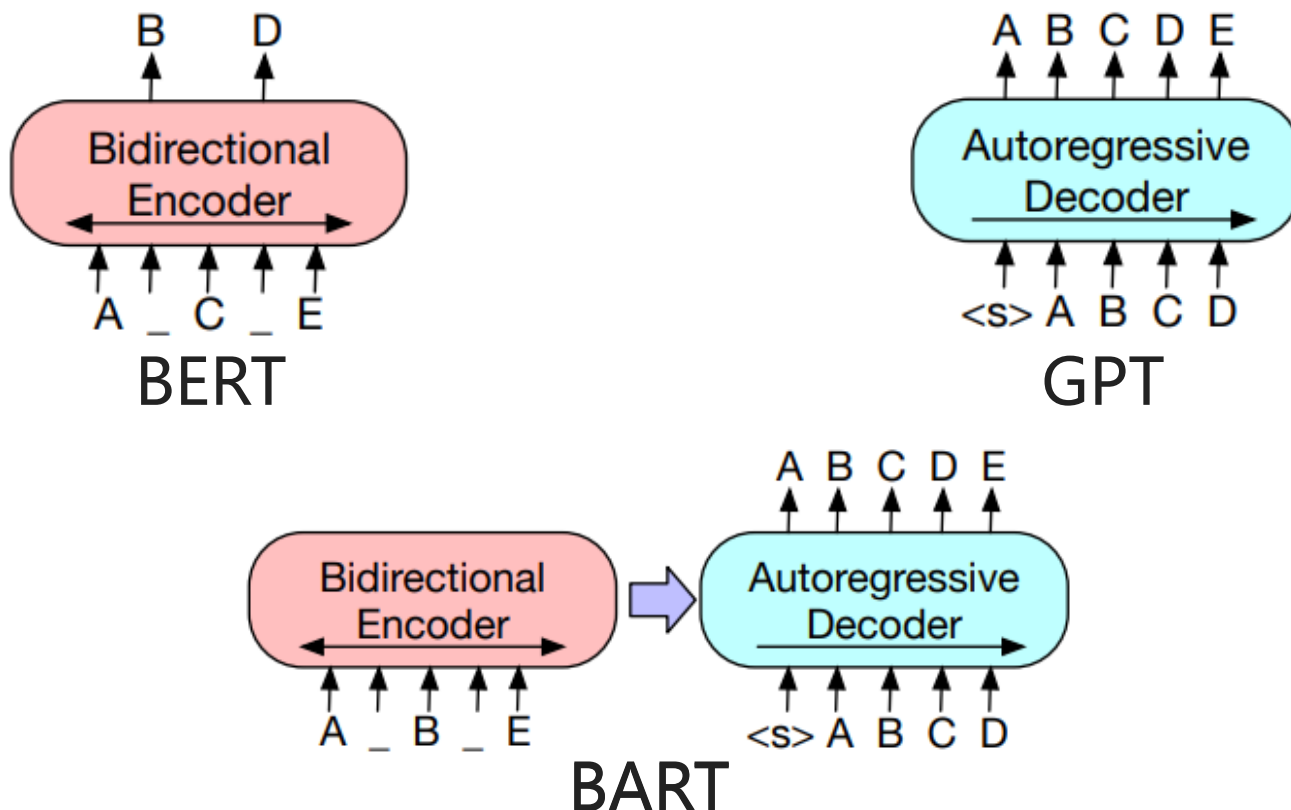
Radford, Alec, et al. "Improving language understanding by generative pre-training." (2018).

单向自回归模型：GPT

- ▶ 直接采用LM任务预训练，不存在[MASK]引入的偏差
- ▶ 在自然语言生成的任务上表现突出
- ▶ 由于LM任务的限制，只能建模上文信息，不能得到严格意义上的上下文相关词向量
- ▶ 不适合自然语言理解相关任务

编解码模型：BART

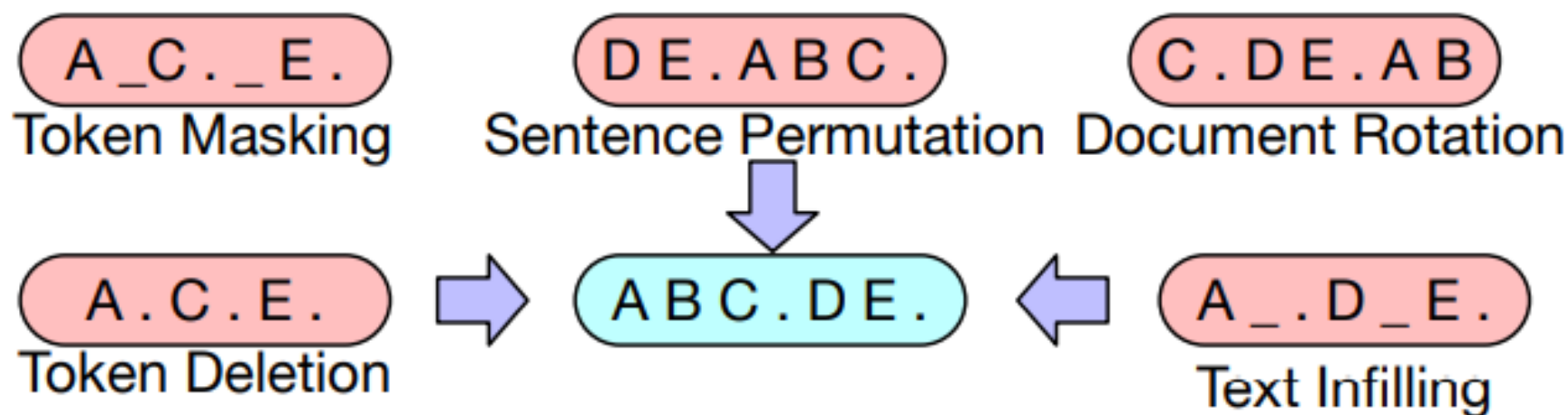
- ▶ 采用完整的Transformer结构
- ▶ 采用MLM来解决Encoder端信息泄露问题



Lewis, Mike, et al. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension." Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020.

编解码模型：BART

- ▶ 采用完整的Transformer结构
- ▶ 采用MLM来解决Encoder端信息泄露问题
- ▶ 引入多种噪音来加强模型泛化性能



Lewis, Mike, et al. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension." Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020.

- ▶ XLNet
 - ▶ Permuted LM
- ▶ UniLM
 - ▶ Prefix LM

- ▶ 自编码模型：
 - ▶ 可以很好的建模双向上下文信息
 - ▶ 但[MASK]的引入：预训练-微调偏差；独立性假设

$$\max_{\theta} \log p_{\theta}(\bar{\mathbf{x}} | \hat{\mathbf{x}}) \approx \sum_{t=1}^T m_t \log p_{\theta}(x_t | \hat{\mathbf{x}}) = \sum_{t=1}^T m_t \log \frac{\exp(H_{\theta}(\hat{\mathbf{x}})_t^{\top} e(x_t))}{\sum_{x'} \exp(H_{\theta}(\hat{\mathbf{x}})_t^{\top} e(x'))},$$

- ▶ 自回归模型：
 - ▶ 没有偏差和假设，严格按照联合概率公式展开

$$\max_{\theta} \log p_{\theta}(\mathbf{x}) = \sum_{t=1}^T \log p_{\theta}(x_t | \mathbf{x}_{<t}) = \sum_{t=1}^T \log \frac{\exp(h_{\theta}(\mathbf{x}_{1:t-1})^{\top} e(x_t))}{\sum_{x'} \exp(h_{\theta}(\mathbf{x}_{1:t-1})^{\top} e(x'))},$$

- ▶ 但不能建模双向上下文信息

- ▶ 怎样同时利用自编码和自回归的优点且克服两者的缺点？

- ▶ 怎样同时利用自编码和自回归的优点且克服两者的缺点？
- ▶ 克服自编码模型的缺点：
 - ▶ 不能引入特殊标签，需要使用LM模式
- ▶ LM模式的核心：
 - ▶ 以“上文”为条件预测下文单词
- ▶ 克服自回归模型的缺点：
 - ▶ 将双向上下文均作为预测内容的条件

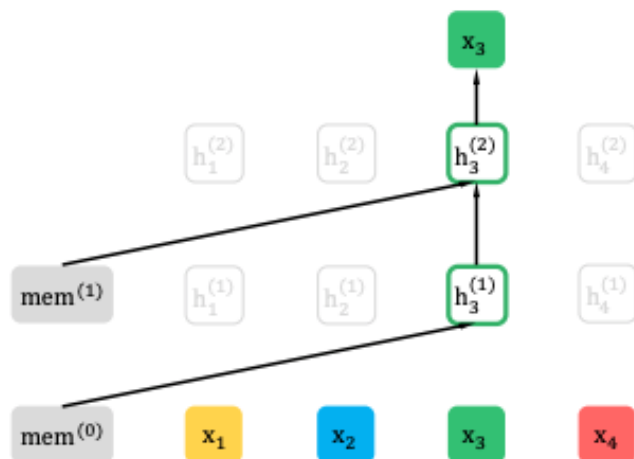
- ▶ 解决方法: Permuted LM
- ▶ 目标函数:

$$\max_{\theta} \mathbb{E}_{\mathbf{z} \sim \mathcal{Z}_T} \left[\sum_{t=1}^T \log p_{\theta}(x_{z_t} \mid \mathbf{x}_{\mathbf{z}_{<t}}) \right].$$

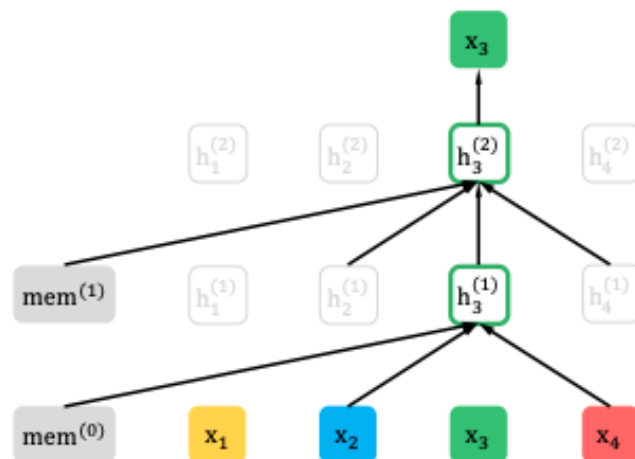
- ▶ 其中:
- ▶ $\mathcal{Z}_T$ 指长度为 T 的序列的所有可能排列方式
- ▶ z_t 指排列 $z \in \mathcal{Z}_T$ 中的第 t 个元素
- ▶ $z_{<t}$ 指排列 $z \in \mathcal{Z}_T$ 中前 $t - 1$ 个元素

其他典型结构

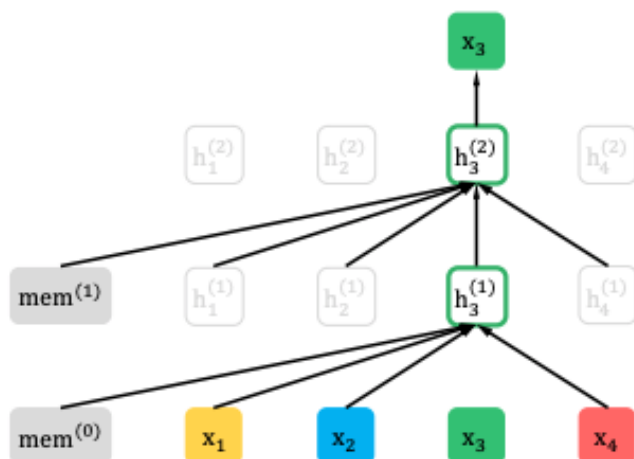
XLNet: Permuted LM



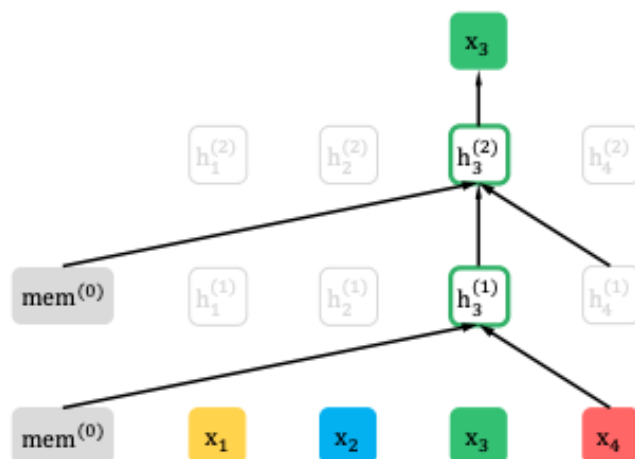
Factorization order: $3 \rightarrow 2 \rightarrow 4 \rightarrow 1$



Factorization order: $2 \rightarrow 4 \rightarrow 3 \rightarrow 1$



Factorization order: $1 \rightarrow 4 \rightarrow 2 \rightarrow 3$



Factorization order: $4 \rightarrow 3 \rightarrow 1 \rightarrow 2$

其他典型结构

XLNet: Permuted LM

- ▶ 示例: New York is a city.
- ▶ 考虑两个排列: $\begin{cases} 3, 4, 5, 1, 2 \\ 3, 4, 5, 2, 1 \end{cases}$
- ▶ 出问题了!

其他典型结构

XLNet: Permuted LM

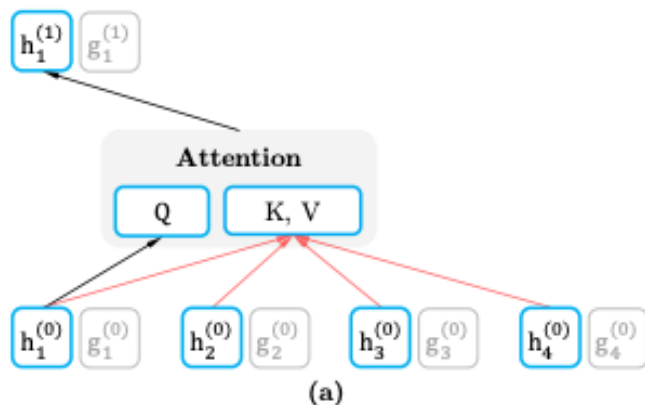
- ▶ 示例: New York is a city.
- ▶ 考虑两个排列: $\begin{cases} 3, 4, 5, 1, 2 \\ 3, 4, 5, 2, 1 \end{cases}$
- ▶ 最终的分布应和预测的词元原本位置有关

其他典型结构

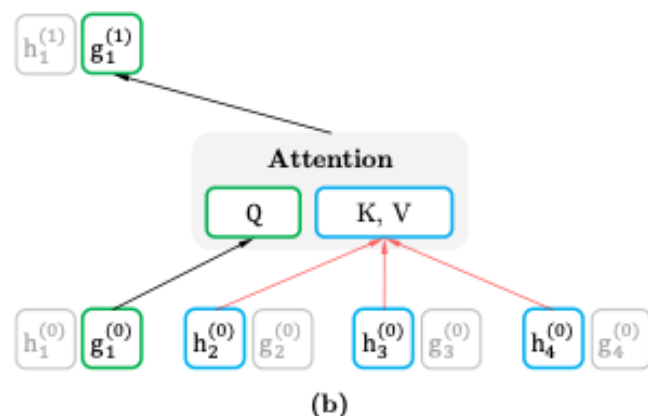
XLNet: Permuted LM

- ▶ 引入Two-Stream Self-Attention
 - ▶ 内容流: $h_{z_t} = h_{\theta}(\mathbf{x}_{z_{\leq t}})$
 - ▶ 查询流: $g_{z_t} = g_{\theta}(\mathbf{x}_{z_{< t}}, z_t)$
- ▶ 最终使用查询流结果生成预测

Sample a factorization order:
 $3 \rightarrow 2 \rightarrow 4 \rightarrow 1$

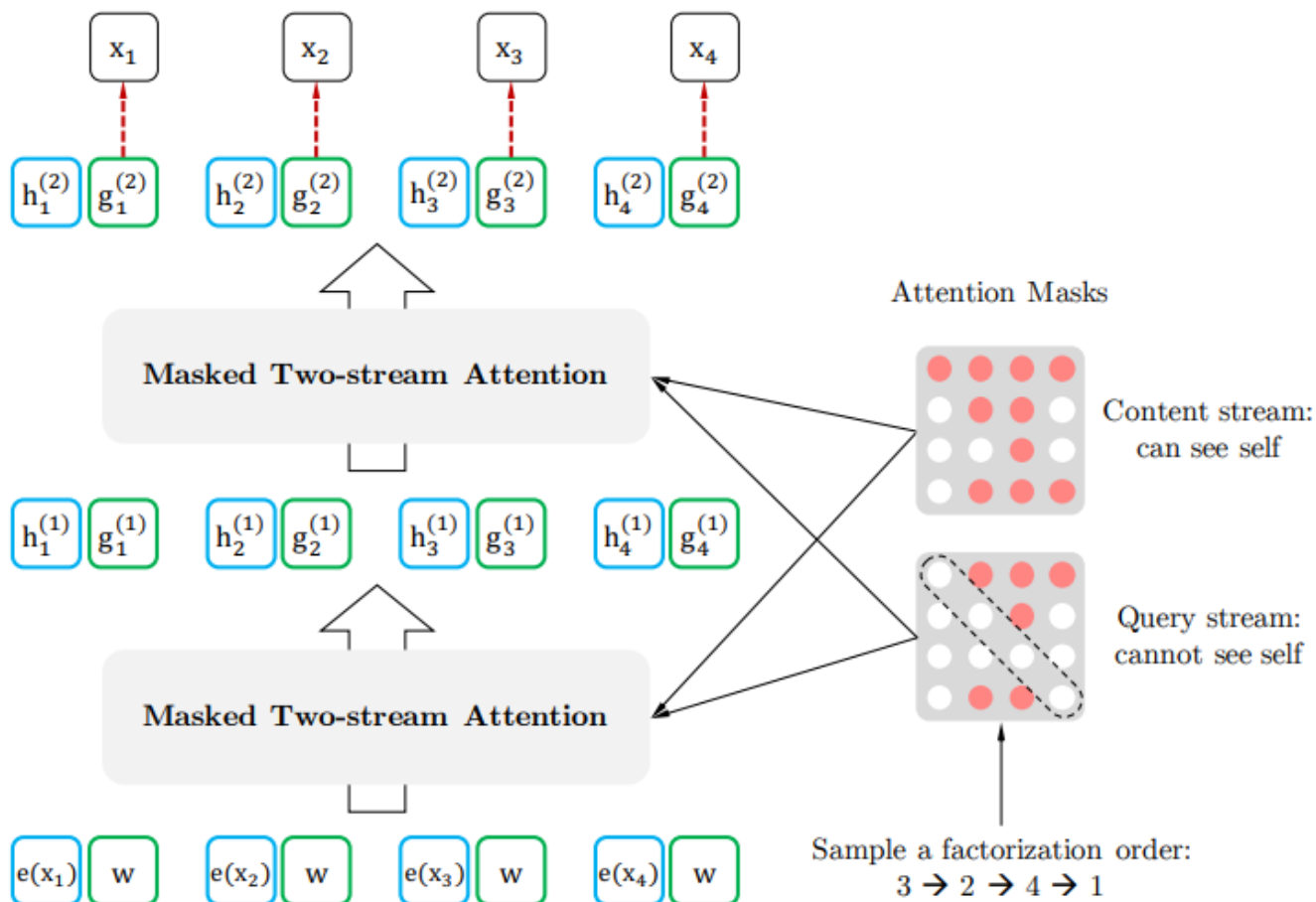


(a) Content stream attention



(b) Query stream attention

► 最终实现：掩码矩阵



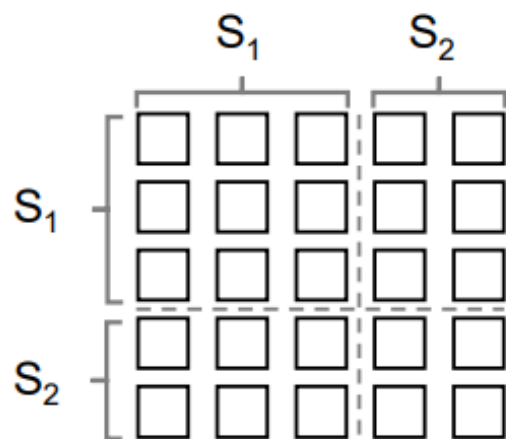
- ▶ 优点：
 - ▶ 成功的结合了自编码和自回归两种模型的优点并避免了两者的缺点
 - ▶ 在很多任务上超越了GPT、BERT等模型
- ▶ 缺点：
 - ▶ 需要遍历所有可能排列，计算量巨大
 - ▶ 采样部分排列进行训练
 - ▶ 收敛速度缓慢
 - ▶ 引入超参数 K ，只考虑每个排列的最后 $1/K$ 的单词作为优化目标

- ▶ 自编码模型更适应自然语言理解任务
- ▶ 自回归模型更适应自然语言生成任务
- ▶ 是否有一个统一的(Unified)模型能够同时适应两种任务？

其他典型结构

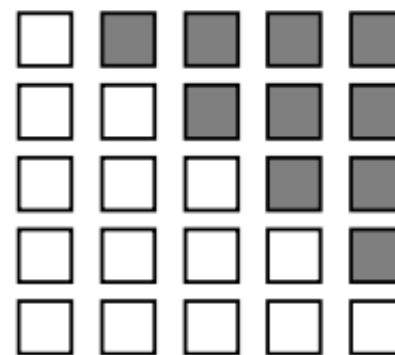
UniLM: Prefix LM

- 从实现角度考虑：两种模型的区别仅在掩码矩阵



S_1 & S_2 : attend to all tokens

自编码模型的掩码矩阵



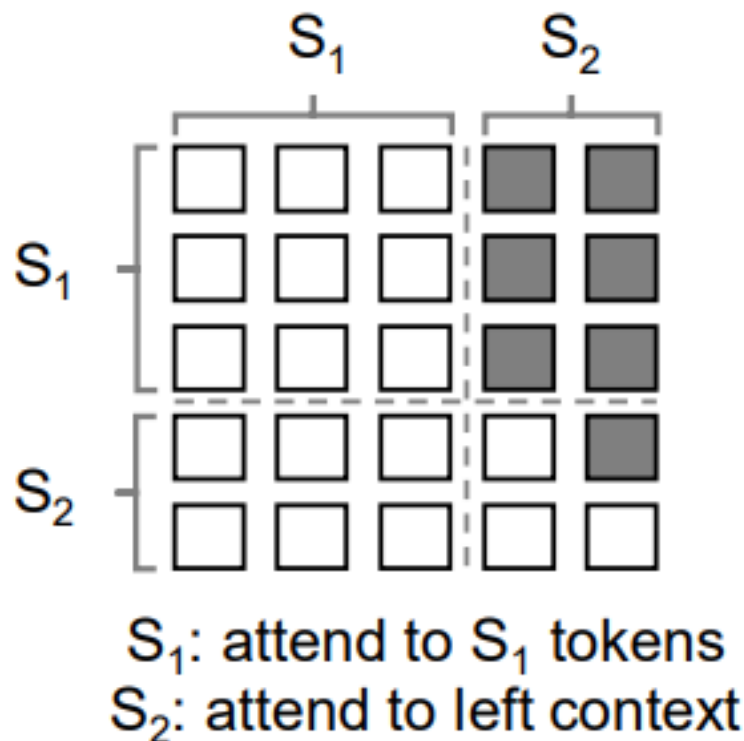
S_1 : attend to left context

自回归模型的掩码矩阵

其他典型结构

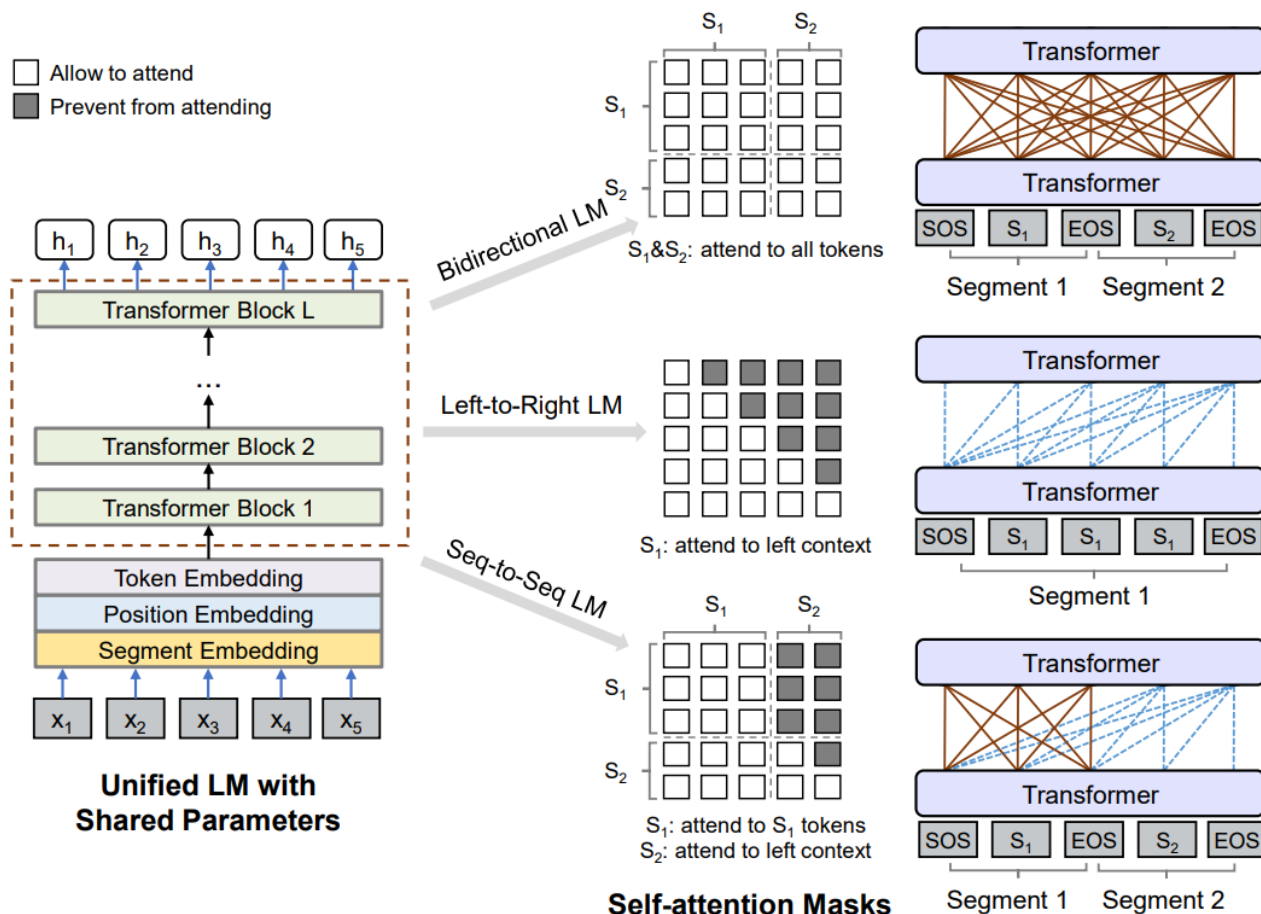
UniLM: Prefix LM

- ▶ 从实现角度考虑：两种模型的区别仅在掩码矩阵
- ▶ 进一步观察：编解码模型也可以统一到这个框架中



Dong, Li, et al. "Unified language model pre-training for natural language understanding and generation." Proceedings of the 33rd International Conference on Neural Information Processing Systems. 2019.

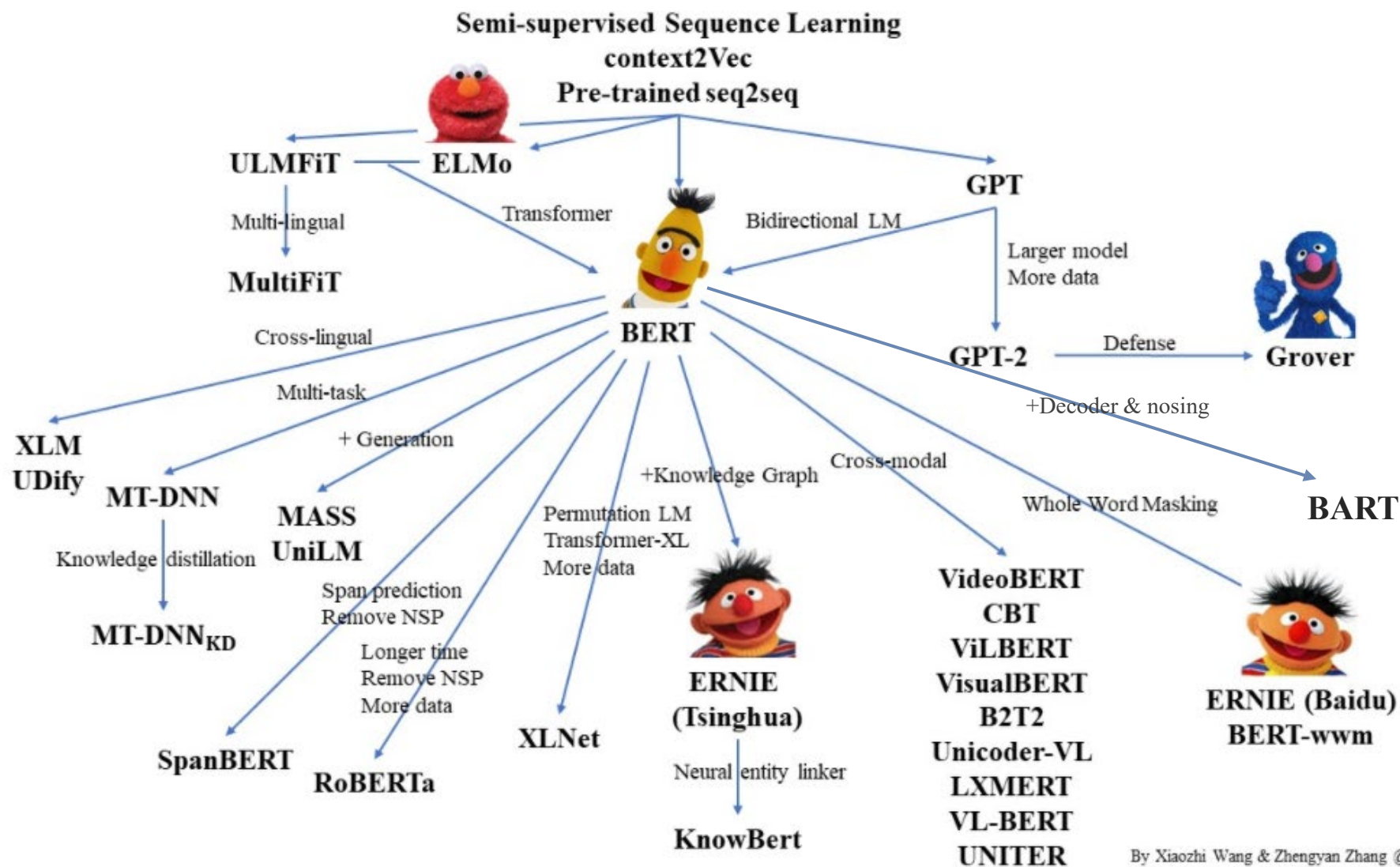
► 统一两种任务的关键：巧用掩码矩阵



Dong, Li, et al. "Unified language model pre-training for natural language understanding and generation." Proceedings of the 33rd International Conference on Neural Information Processing Systems. 2019.

- ▶ 使用 $BERT_{large}$ 参数初始化
- ▶ 使用四个任务继续预训练
 - ▶ 单向LM：结合了MLM和LM两个任务，使用[MASK]词元来掩蔽目标词元，其余和语言模型相同
 - ▶ 双向LM：与BERT中MLM任务相同
 - ▶ 序列到序列LM：对于两句话的输入，前一句话使用双向LM任务，后一句话使用单向LM任务
 - ▶ NSP：和BERT中NSP任务相同

典型预训练语言模型总结



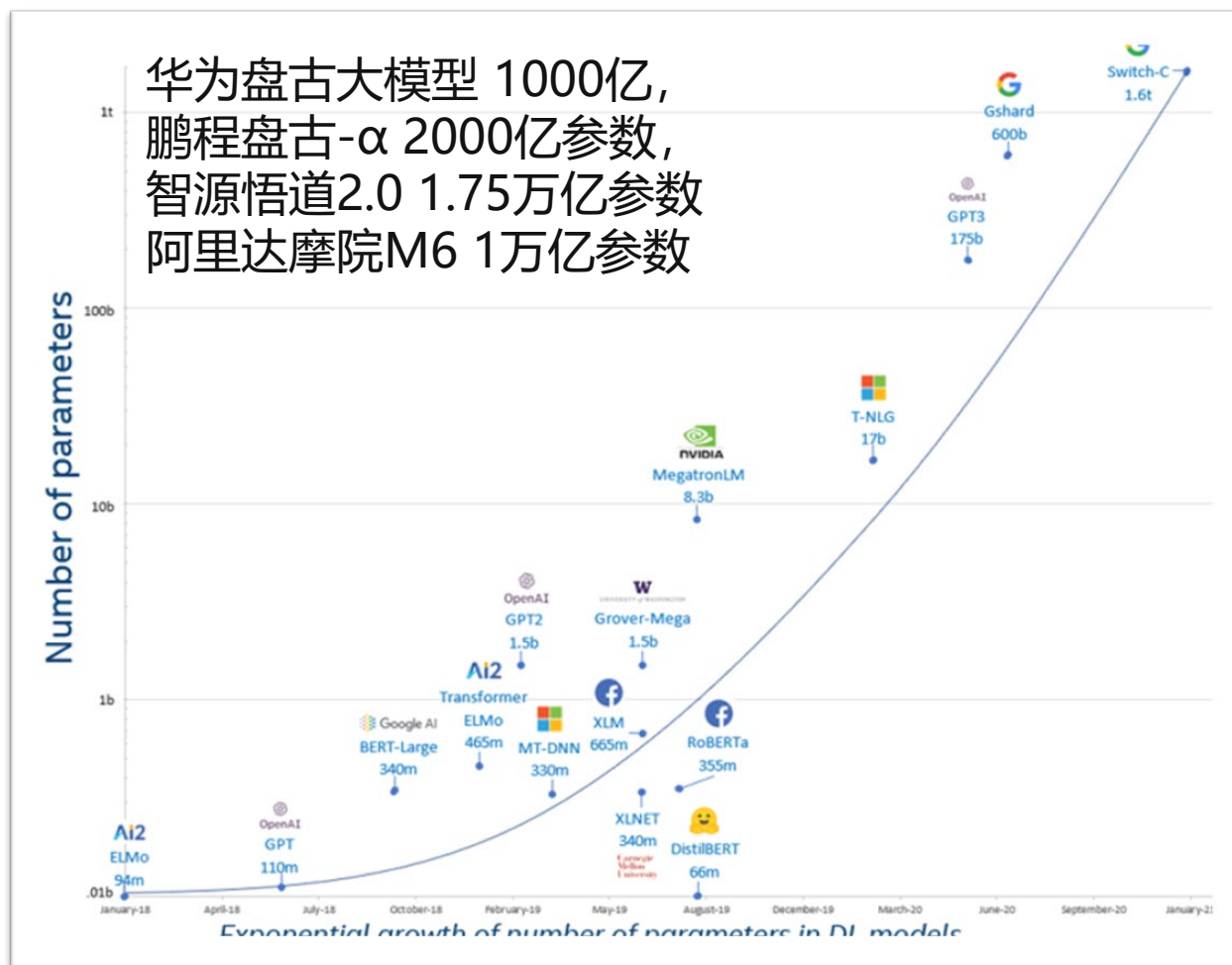
By Xiaozhi Wang & Zhengyan Zhang @THUNLP

预训练语言模型的发展

- ▶ 模型自身结构的改进
- ▶ 优化预训练任务
- ▶ 预训练语料来源及其组成
- ▶ 微调方法

预训练语言模型的发展

► 不断扩大的参数量与计算量



From Daxin Jiang's PPT

预训练语言模型的发展

模型自身结构的改进：模型的压缩与简化

► ALBERT

► 核心思想

- 嵌入分解：将嵌入层分解为两个更小的矩阵的乘积
- 参数共享：将所有层分成多个组，每组的参数完全共享
- 极大的减少了参数量，但是计算量同时增加

Model		Parameters	SQuAD1.1	SQuAD2.0	MNLI	SST-2	RACE	Avg	Speedup
BERT	base	108M	90.4/83.2	80.4/77.6	84.5	92.8	68.2	82.3	4.7x
	large	334M	92.2/85.5	85.0/82.2	86.6	93.0	73.9	85.2	1.0
ALBERT	base	12M	89.3/82.3	80.0/77.1	81.6	90.3	64.0	80.1	5.6x
	large	18M	90.6/83.9	82.3/79.4	83.5	91.7	68.5	82.4	1.7x
	xlarge	60M	92.5/86.1	86.1/83.1	86.4	92.4	74.8	85.5	0.6x
	xxlarge	235M	94.1/88.3	88.1/85.1	88.0	95.2	82.3	88.7	0.3x

Table 2: Dev set results for models pretrained over BOOKCORPUS and Wikipedia for 125k steps. Here and everywhere else, the Avg column is computed by averaging the scores of the downstream tasks to its left (the two numbers of F1 and EM for each SQuAD are first averaged).

预训练语言模型的发展

模型自身结构的改进：模型的压缩与简化

- ▶ 知识蒸馏(+剪枝): DistillBERT、TinyBERT等
- ▶ 量化: Q-BERT、Q8BERT等

Table 3: Comparison of Compressed PTMs

Method	Type	#Layer	Loss Function*	Speed Up	Params	Source PTM	GLUE [‡]
BERT _{BASE} [36]	Baseline	12	$\mathcal{L}_{MLM} + \mathcal{L}_{NSP}$		110M		79.6
BERT _{LARGE} [36]		24	$\mathcal{L}_{MLM} + \mathcal{L}_{NSP}$		340M		81.9
Q-BERT [156]	Quantization	12	HAWQ + GWQ	-		BERT _{BASE}	$\approx 99\%$ BERT [°]
Q8BERT [211]		12	DQ + QAT	-		BERT _{BASE}	$\approx 99\%$ BERT
ALBERT [§] [93]	Param. Sharing	12	$\mathcal{L}_{MLM} + \mathcal{L}_{SOP}$	$\times 5.6 \sim 0.3$	12 ~ 235M		89.4 (ensemble)
DistilBERT [152]	Distillation	6	$\mathcal{L}_{KD-CE} + \text{Cos}_{KD} + \mathcal{L}_{MLM}$	$\times 1.63$	66M	BERT _{BASE}	77.0 (dev)
TinyBERT ^{§†} [75]		4	$\text{MSE}_{\text{embed}} + \text{MSE}_{\text{attn}} + \text{MSE}_{\text{hidn}} + \mathcal{L}_{KD-CE}$	$\times 9.4$	14.5M	BERT _{BASE}	76.5
BERT-PKD [169]		3 ~ 6	$\mathcal{L}_{KD-CE} + \text{PT}_{KD} + \mathcal{L}_{\text{Task}}$	$\times 3.73 \sim 1.64$	45.7 ~ 67 M	BERT _{BASE}	76.0 ~ 80.6 [#]
PD [183]		6	$\mathcal{L}_{KD-CE} + \mathcal{L}_{\text{Task}} + \mathcal{L}_{MLM}$	$\times 2.0$	67.5M	BERT _{BASE}	81.2 [#]
MobileBERT [§] [172]		24	$\text{FMT} + \text{AT} + \text{PKT} + \mathcal{L}_{KD-CE} + \mathcal{L}_{MLM}$	$\times 4.0$	25.3M	BERT _{LARGE}	79.7
MiniLM [194]		6	AT+AR	$\times 1.99$	66M	BERT _{BASE}	81.0 ^b
DualTrain ^{§†} [216]		12	Dual Projection + $\mathcal{L}_{MLM}$	-	1.8 ~ 19.2M	BERT _{BASE}	75.8 ~ 81.9 ^h
BERT-of-Theseus [203]	Module Replacing	6	$\mathcal{L}_{\text{Task}}$	$\times 1.94$	66M	BERT _{BASE}	78.6

预训练语言模型的发展

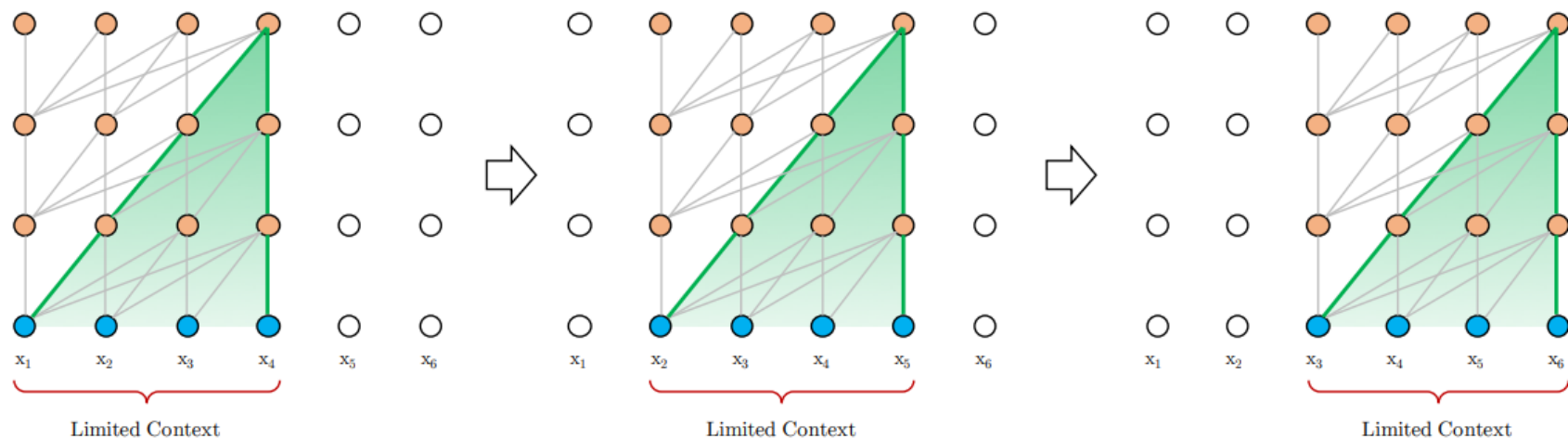
模型自身结构的改进：处理长文本问题

- ▶ 由于显存限制和训练效率要求，预训练阶段输入句子的长度不能过长；
- ▶ 由于Transformer对于顺序的不敏感，需要特别通过Position Embedding来引入位置信息。
- ▶ 使用时输入长度受限于预训练文本长度（一般为512）

预训练语言模型的发展

模型自身结构的改进：处理长文本问题

- ▶ 一个简易的解决方案：
 - ▶ 滑动窗口



Dai, Zihang, et al. "Transformer-XL: Attentive Language Models beyond a Fixed-Length Context." Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. 2019.

预训练语言模型的发展

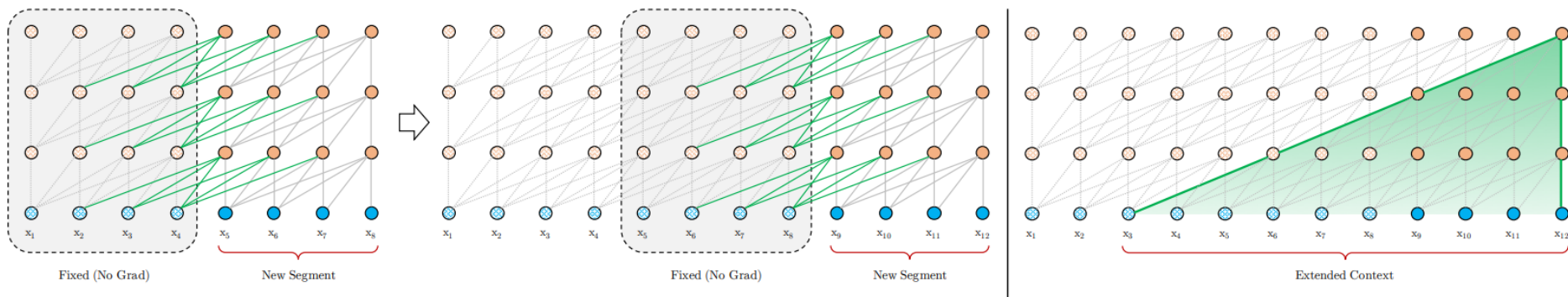
模型自身结构的改进：处理长文本问题

- ▶ 一个简易的解决方案：
 - ▶ 滑动窗口
- ▶ 问题：
 - ▶ 类似于固定窗口语言模型，限制了最大的依赖长度
 - ▶ 有大量的重复计算，浪费计算资源

预训练语言模型的发展

模型自身结构的改进：处理长文本问题

- ▶ Transformer-XL
 - ▶ Segment-level Recurrent



Dai, Zihang, et al. "Transformer-XL: Attentive Language Models beyond a Fixed-Length Context." Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. 2019.

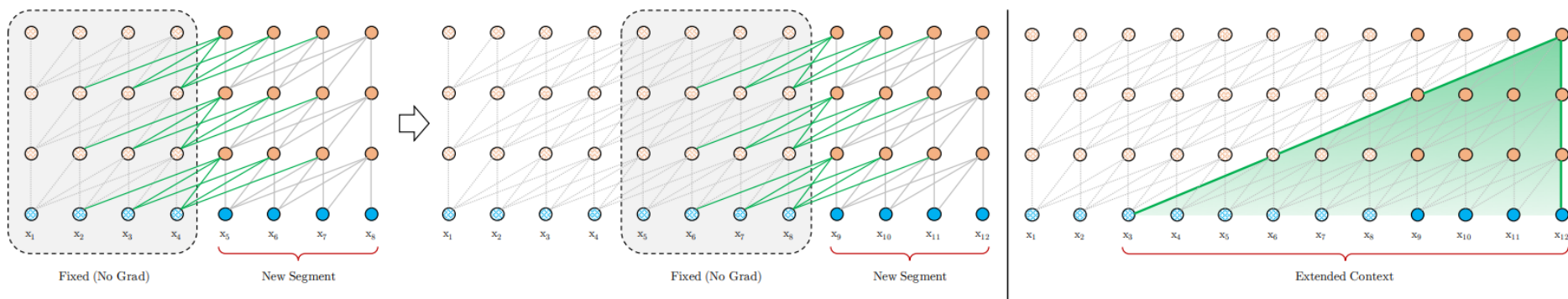
预训练语言模型的发展

模型自身结构的改进：处理长文本问题

► Transformer-XL

► Segment-level Recurrent

- 将上一个Segment计算时的隐状态缓存下来
- 计算后一个Segment时使用缓存隐状态来增大上下文范围
- 缓存隐状态不参与梯度反传



Dai, Zihang, et al. "Transformer-XL: Attentive Language Models beyond a Fixed-Length Context." Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. 2019.

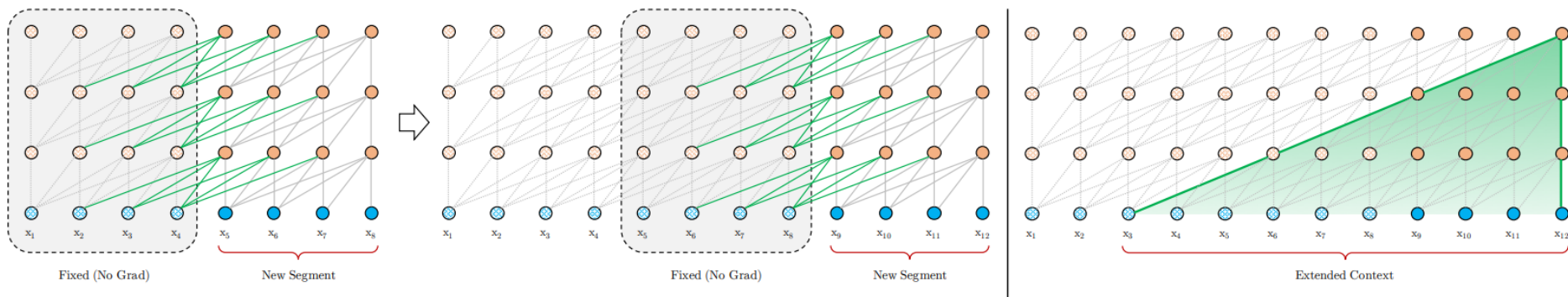
预训练语言模型的发展

模型自身结构的改进：处理长文本问题

► Transformer-XL

► Segment-level Recurrent

- 将上一个Segment计算时的隐状态缓存下来
- 计算后一个Segment时使用缓存隐状态来增大上下文范围
- 缓存隐状态不参与梯度反传
- 问题：能建模的上下文范围扩大了多少？



Dai, Zihang, et al. "Transformer-XL: Attentive Language Models beyond a Fixed-Length Context." Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. 2019.

预训练语言模型的发展

优化预训练任务

- ▶ ALBERT: NSP -> SOP
 - ▶ 句子顺序预测 (Sentence Order Prediction)
 - ▶ 预测两句话的顺序
 - ▶ 排除原来可能有的主题等其他因素
- ▶ Electra: +RTD
 - ▶ 替换词元检测 (Replaced Token Detection)

预训练语言模型的发展

优化预训练任务：RTD

- ▶ 回忆BERT中[MASK]面临的Trade-off:
 - ▶ 掩蔽过多词元——破坏上下文信息
 - ▶ 掩蔽过少词元——训练缓慢，难以收敛

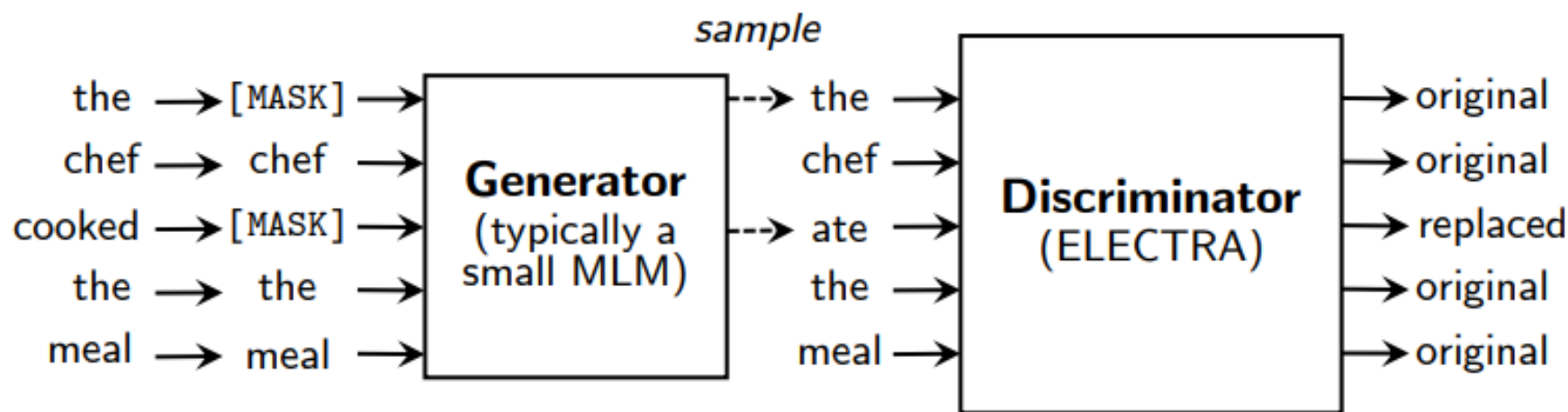
预训练语言模型的发展

优化预训练任务：RTD

- ▶ 回忆BERT中[MASK]面临的Trade-off:
 - ▶ 掩蔽过多词元——破坏上下文信息
 - ▶ 掩蔽过少词元——训练缓慢，难以收敛
- ▶ 受启发于GAN的思想——用一个鉴别器来检测每个词元是否是原句中的词元。

预训练语言模型的发展

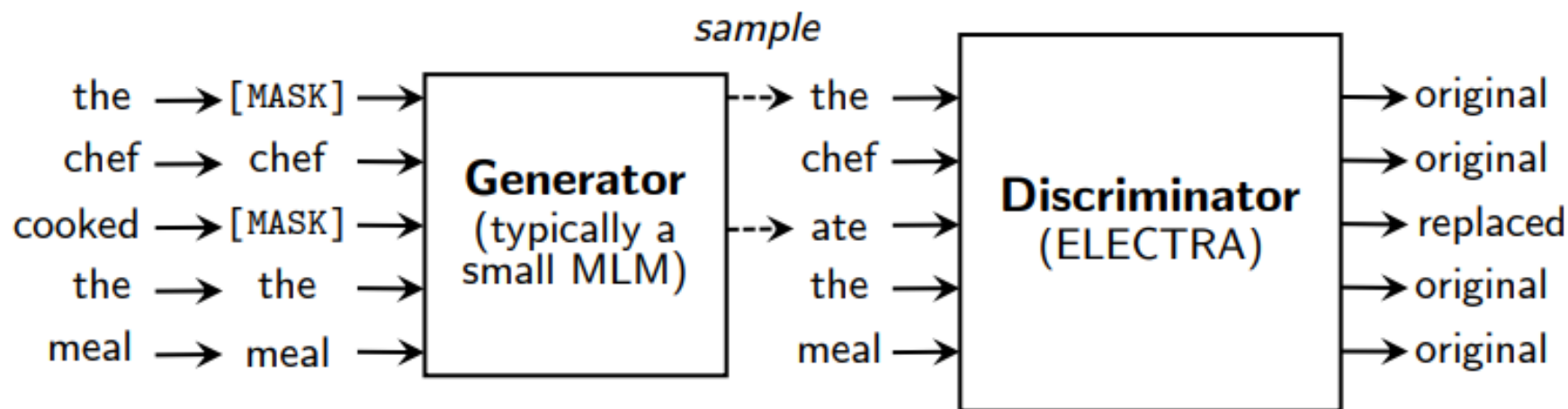
优化预训练任务：RTD



Clark, Kevin, et al. "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators." International Conference on Learning Representations. 2019.

预训练语言模型的发展

优化预训练任务：RTD

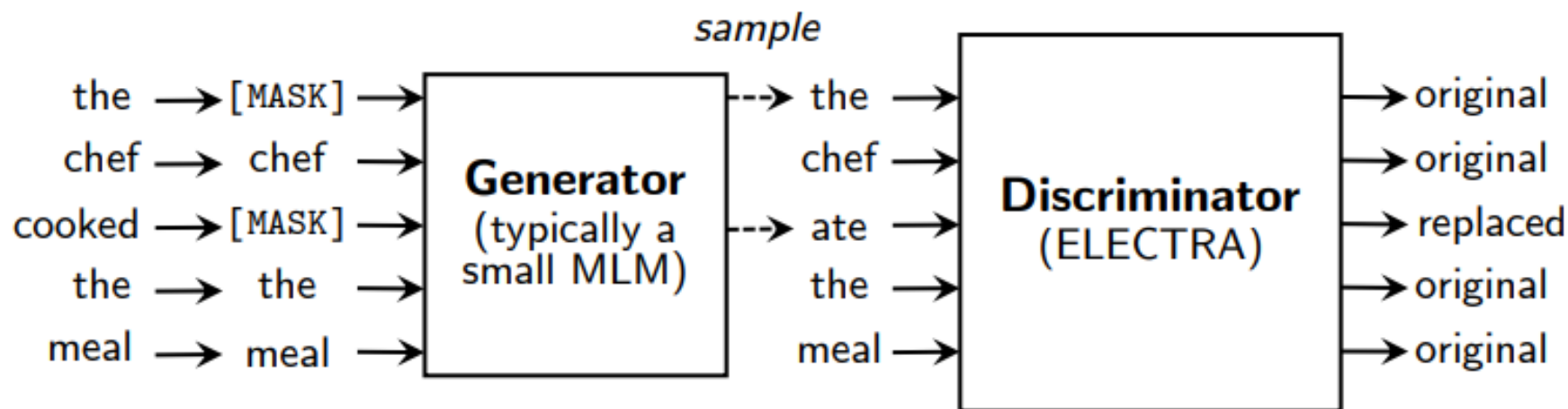


► 优势：

- 极大的提高了训练的效率
- 避免了因为[MASK]带来的预训练-微调偏差

预训练语言模型的发展

优化预训练任务：Electra



► 理解生成器的作用：

- 对随机掩蔽的词元进行筛选，提高任务的难度
- 实验发现使用较小的生成器效果更好
 - 如果最终任务过难可能导致Electra不能进行有效学习

预训练语言模型的发展

优化预训练任务：Electra

Model	Train / Infer FLOPs	Speedup	Params	Train Time + Hardware	GLUE
ELMo	3.3e18 / 2.6e10	19x / 1.2x	96M	14d on 3 GTX 1080 GPUs	71.2
GPT	4.0e19 / 3.0e10	1.6x / 0.97x	117M	25d on 8 P6000 GPUs	78.8
BERT-Small	1.4e18 / 3.7e9	45x / 8x	14M	4d on 1 V100 GPU	75.1
BERT-Base	6.4e19 / 2.9e10	1x / 1x	110M	4d on 16 TPUv3s	82.2
ELECTRA-Small	1.4e18 / 3.7e9	45x / 8x	14M	4d on 1 V100 GPU	79.9
50% trained	7.1e17 / 3.7e9	90x / 8x	14M	2d on 1 V100 GPU	79.0
25% trained	3.6e17 / 3.7e9	181x / 8x	14M	1d on 1 V100 GPU	77.7
12.5% trained	1.8e17 / 3.7e9	361x / 8x	14M	12h on 1 V100 GPU	76.0
6.25% trained	8.9e16 / 3.7e9	722x / 8x	14M	6h on 1 V100 GPU	74.1
ELECTRA-Base	6.4e19 / 2.9e10	1x / 1x	110M	4d on 16 TPUv3s	85.1

Model	Train FLOPs	Params	CoLA	SST	MRPC	STS	QQP	MNLI	QNLI	RTE	Avg.
BERT	1.9e20 (0.27x)	335M	60.6	93.2	88.0	90.0	91.3	86.6	92.3	70.4	84.0
RoBERTa-100K	6.4e20 (0.90x)	356M	66.1	95.6	91.4	92.2	92.0	89.3	94.0	82.7	87.9
RoBERTa-500K	3.2e21 (4.5x)	356M	68.0	96.4	90.9	92.1	92.2	90.2	94.7	86.6	88.9
XLNet	3.9e21 (5.4x)	360M	69.0	97.0	90.8	92.2	92.3	90.8	94.9	85.9	89.1
BERT (ours)	7.1e20 (1x)	335M	67.0	95.9	89.1	91.2	91.5	89.6	93.5	79.5	87.2
ELECTRA-400K	7.1e20 (1x)	335M	69.3	96.0	90.6	92.1	92.4	90.5	94.5	86.8	89.0
ELECTRA-1.75M	3.1e21 (4.4x)	335M	69.1	96.9	90.8	92.6	92.4	90.9	95.0	88.0	89.5

Clark, Kevin, et al. "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators." International Conference on Learning Representations. 2019.

预训练语言模型的发展

预训练语料来源及其组成

- ▶ 多语言
- ▶ 多模态
- ▶ 领域特定与知识增强

预训练语言模型的发展

预训练语料来源及其组成：多语言

- ▶ 核心问题：
 - ▶ 使用统一的模型对多种语言建模——多语言理解
 - ▶ 多语言的翻译和对齐——多语言生成

预训练语言模型的发展

预训练语料来源及其组成：多语言

- ▶ 核心问题：
 - ▶ 使用统一的模型对多种语言建模——多语言理解
 - ▶ 多语言的翻译和对齐——多语言生成
- ▶ 多语言理解：
 - ▶ mBERT——简单的使用多种语言语料进行预训练
 - ▶ XLM——在mBERT基础上，引入了机器翻译这个监督任务辅助
 - ▶ Unicoder——在XLM基础上，引入不同粒度的对齐任务（跨语言词语恢复，跨语言句义匹配判别，跨语言MLM），仍需要监督数据

预训练语言模型的发展

预训练语料来源及其组成：多语言

- ▶ 核心问题：
 - ▶ 使用统一的模型对多种语言建模——多语言理解
 - ▶ 多语言的翻译和对齐——多语言生成
- ▶ 多语言理解：
 - ▶ mBERT、XLM、Unicoder等
- ▶ 多语言生成：
 - ▶ MASS——模型结构类似BART，使用多种语言的单语言语料进行训练
 - ▶ XNLG—— 2×2 ，两阶段预训练（encoder-MLM, decoder-DAE），单语言和跨语言两种语料

预训练语言模型的发展

预训练语料来源及其组成：多模态

- ▶ 图像+文本：按照两个模态的交互位置分类
 - ▶ 双流（分别编码后交互再编码）
 - ▶ ViBERT(MMM, MMA), LXMBERT(MLM, RoIFR, DLC, CMM, IQA)等
 - ▶ 单流（直接统一进行编码）
 - ▶ VisualBERT(MLM, SIP), VL-BERT(MLM, MRoIC)等
- ▶ 视频+文本
 - ▶ VideoBERT(MLM, MFM), UniVL(VTJ, CMLM, CMFM, VTA, LR)等
- ▶ 音频+文本
 - ▶ SpeechBERT(After Initial Phonetic-Semantic Joint Embedding, MLM & MAL)等

预训练语言模型的发展

预训练语料来源及其组成：领域特定与知识增强

- ▶ 领域特定语料预训练
 - ▶ 面向专业领域：
 - ▶ BioBERT, SciBERT, ClinicalBERT等
 - ▶ 面向特定下游任务：
 - ▶ 情绪分析——Label Aware MLM
 - ▶ 文本总结——Gap Sentence Generation
 - ▶ 不流畅语流检测——Noisy Words Detection

预训练语言模型的发展

预训练语料来源及其组成：领域特定与知识增强

- ▶ 知识嵌入增强
 - ▶ (词元级别) 语言学知识
 - ▶ 例如词性标签、情感极性 etc
 - ▶ SentiLR、SenceBERT等
 - ▶ (实体级别) 知识图谱得到的实体嵌入
 - ▶ 例如将实体嵌入与文本中相关实体的词向量进行融合
 - ▶ ERNIE (THU)、KnowBERT、KEPLER等
 - ▶ (句子级别) 知识图谱中的关系三元组
 - ▶ 例如利用三元组拓展句子成树后进行编码
 - ▶ K-BERT等

- ▶ 提升在特定下游任务上的性能
 - ▶ 两阶段微调 (Two-Stage)
 - ▶ 先在其他相同领域的数据上预微调
 - ▶ 多任务学习 (Multi Task)
 - ▶ 多个下游任务同时学习
 - ▶ 解冻方法 (Unfreezing)
 - ▶ 由少到多的微调参数，例如逐层解冻

预训练语言模型的发展

微调方法

- ▶ 提升在特定下游任务上的性能
 - ▶ 两阶段微调 (Two-Stage)
 - ▶ 先在其他相同领域的数据上预微调
 - ▶ 多任务学习 (Multi Task)
 - ▶ 多个下游任务同时学习
 - ▶ 解冻方法 (Unfreezing)
 - ▶ 由少到多的微调参数，例如逐层解冻
- ▶ 统一所有的自然语言处理任务
 - ▶ In-context learning

预训练语言模型的发展

In-Context Learning

► GPT-2与GPT的区别

► 更大的模型

参数量	层数	词元嵌入维度
177M (GPT-1)	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

► 更多的数据

- 约800万篇文章、总计40G

预训练语言模型的发展

In-Context Learning

- ▶ GPT-2与GPT的本质区别
 - ▶ 使用无监督的预训练模型做下游任务

预训练语言模型的发展

In-Context Learning

- ▶ GPT-2与GPT的本质区别
 - ▶ 使用无监督的预训练模型不微调做下游任务
 - ▶ 更多的无监督数据、更少的有监督数据

预训练语言模型的发展

In-Context Learning

- ▶ GPT-2的核心思想
 - ▶ 使用无监督的预训练模型不微调做下游任务
 - ▶ 更多的无监督数据、更少的有监督数据

- ▶ 原理

- ▶ 语言模型本质上是在建模

$$P(w_{k+1}, w_{k+2}, \dots, w_N | w_1, w_2, \dots, w_k)$$

- ▶ 而有监督任务实际上实在建模

$$P(\text{Output} | \text{Input})$$

- ▶ 考虑到每个具体任务会使用不同的模型，可以将任务也作为条件，即

$$P(\text{Output} | \text{Input}, \text{Task})$$

Radford, Alec, et al. "Language models are unsupervised multitask learners." OpenAI blog 1.8 (2019): 9.

预训练语言模型的发展

In-Context Learning

- ▶ GPT-2的核心思想

- ▶ 使用无监督的预训练模型不微调做下游任务
- ▶ 更多的无监督数据、更少的有监督数据

- ▶ 原理

- ▶ 语言模型本质上是在建模

$$P(w_{k+1}, w_{k+2}, \dots, w_N | w_1, w_2, \dots, w_k)$$

- ▶ 而有监督任务实际上实在建模

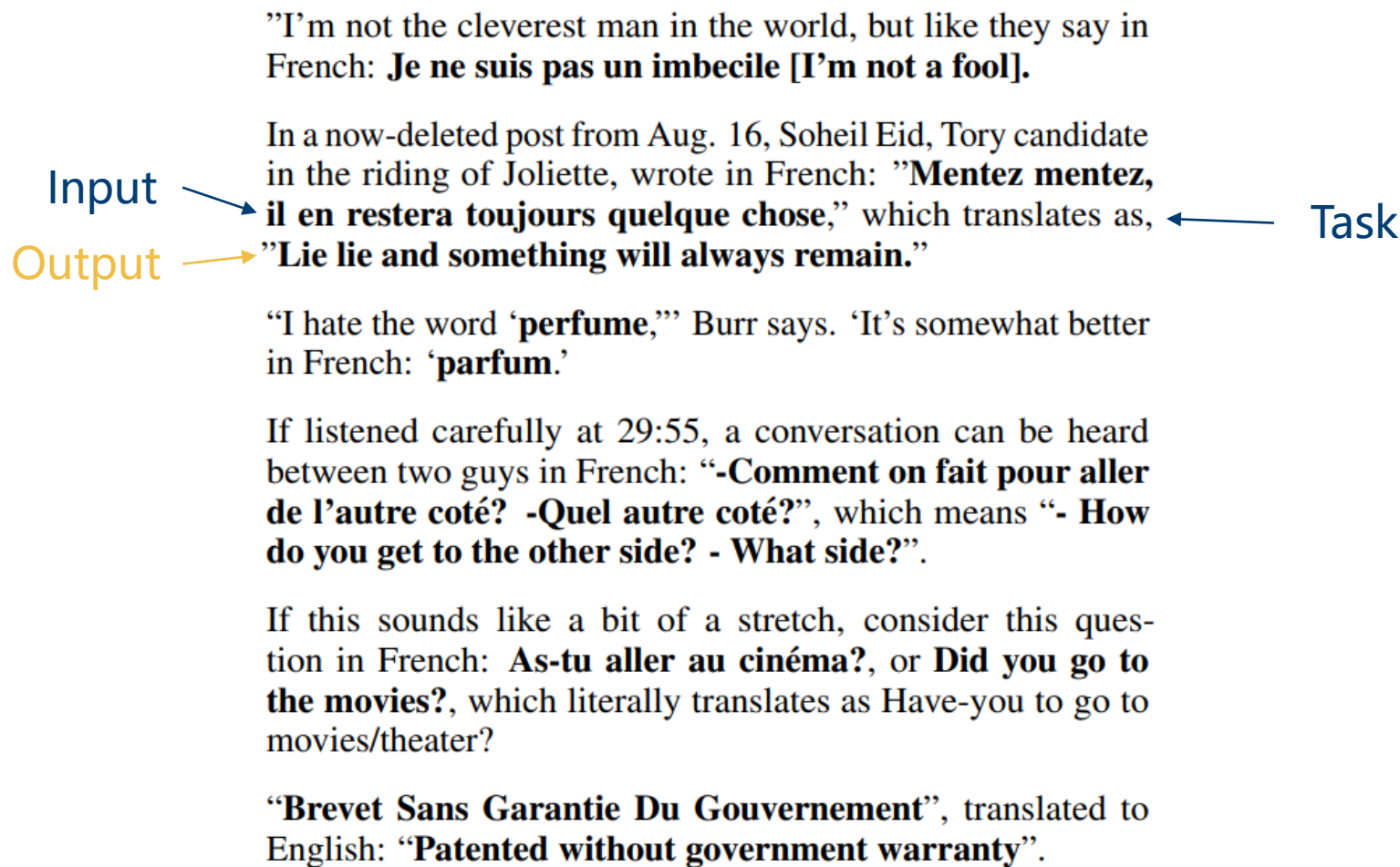
$$P(\text{Output} | \text{Input}, \text{Task})$$

- ▶ 因此有监督任务实际上是无监督语言模型的一种特例
- ▶ 只要模型容量足够大，可以直接通过无监督训练学习到有监督任务

预训练语言模型的发展

In-Context Learning

► 例如在GPT-2的训练集中存在如下数据



Radford, Alec, et al. "Language models are unsupervised multitask learners." OpenAI blog 1.8 (2019): 9.

预训练语言模型的发展

In-Context Learning

- ▶ GPT-3: 进一步明确上述思想
 - ▶ 更更大的模型
 - ▶ GPT-2: 48层、1600维、参数量1.5B
 - ▶ GPT-3: 96层、12888维、序列长度2048、参数量175B
 - ▶ 更更多的数据

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

- ▶ 明确概念: In-Context Learning

► In-Context Learning

► Zero-Shot场景

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

- ▶ In-Context Learning
 - ▶ Zero-Shot场景
 - ▶ One-Shot场景

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← example
3	cheese =>	← prompt

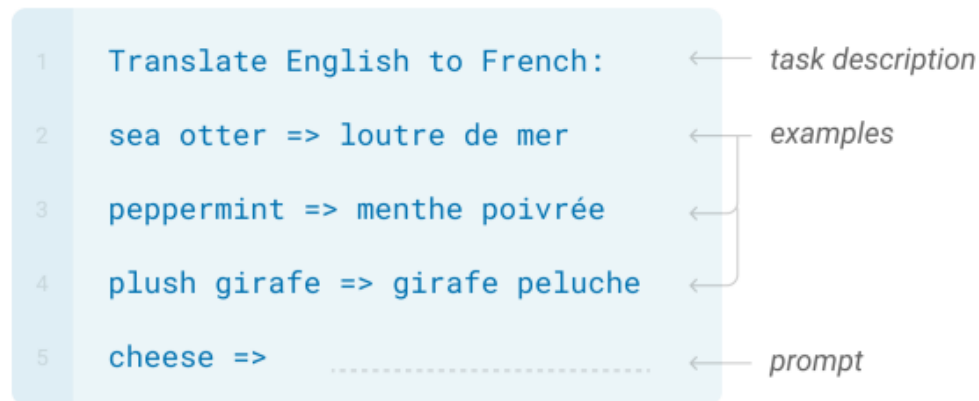
预训练语言模型的发展

In-Context Learning

- ▶ In-Context Learning
 - ▶ Zero-Shot场景
 - ▶ One-Shot场景
 - ▶ Few-Shot场景

Few-shot

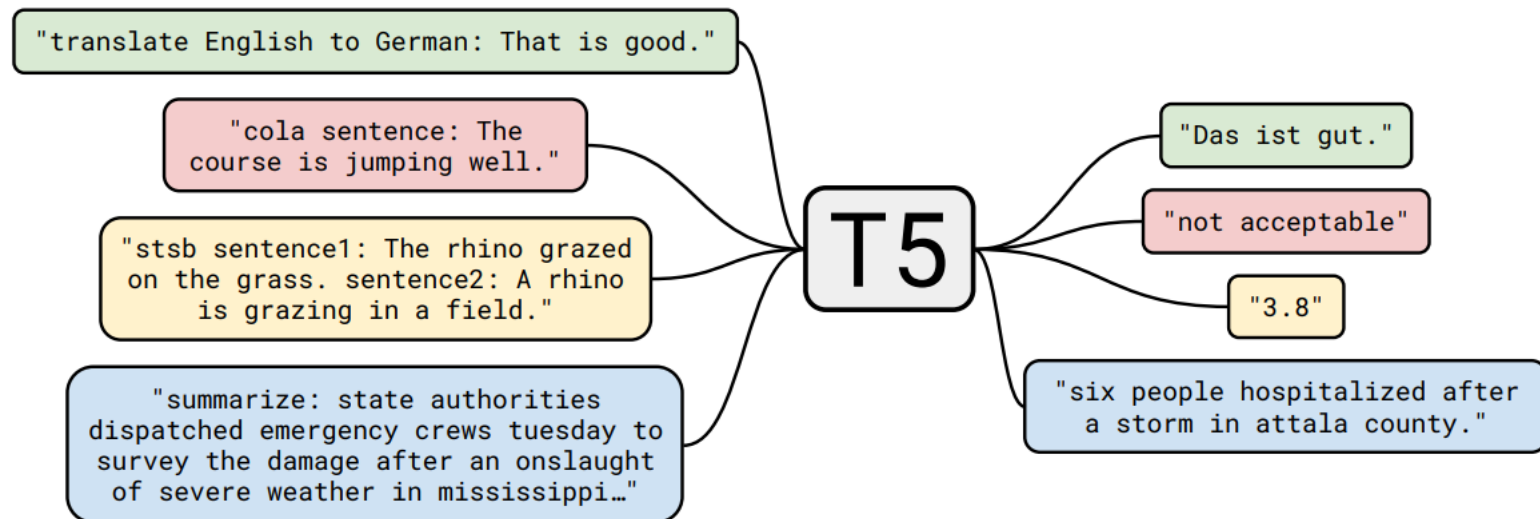
In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



预训练语言模型的发展

In-Context Learning

► 类似的思想：T5



► 进一步的发展：带提示的微调（Prompt）

Raffel, Colin, et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer." Journal of Machine Learning Research 21 (2020): 1-67.

预训练语言模型的发展

ChatGPT

► <https://openai.com/blog/chatgpt/>

USER **this code is not working like i expect — how do i fix it?**

```
resultWorkerErr := make(chan error)
defer close(resultWorkerErr)
go func() {
    defer cancel()
    resultWorkerErr <- b.resultWorker(ctx)
}()

err := b.worker(ctx)
cancel()
if err == nil {
    return <-resultWorkerErr
}
return multierror.Append(err, <-resultWorkerErr)
```

CHATGPT It's difficult to say what's wrong with the code without more context. Can you provide more information about what the code is supposed to do and what isn't working as expected? Also, is this the entire code or just a part of it?

USER **just part of it — the error never surfaces. I think there is something wrong with the channel**